

University of Stuttgart
Germany
Institute of Industrial Automation and
Software Engineering

Probabilistic Action Prediction of Human Drivers Using Dynamic Bayesian Networks

Master Thesis 1111

**Submitted at the University of Stuttgart by
Amal Chulliyat Jose**

Electrical Engineering

Examiner: Prof. Dr.-Ing. Andrey Morozov

Supervisor: Georgios Katranis, M.Sc.

15 April 2026



Contents

List of Figures	v
List of Tables	vi
List of Abbreviations	vii
Glossary	xi
Kurzfassung	xiii
Abstract	xv
1. Introduction	1
2. Theoretical Background	3
2.1. Probability Theory	3
2.1.1. Bayes' Theorem	4
2.2. Probabilistic Graphical Models	4
2.2.1. Bayesian Networks	5
2.2.2. Dynamic Bayesian Networks	6
2.3. Inference in Chain-Structured DBNs	6
2.3.1. Forward-Backward Algorithm	7
2.4. Parameter Learning in Latent-State Models	9
2.4.1. Expectation-Maximization Algorithm	9
3. State of the Art	11
3.1. Introduction	11
3.2. Rule-Based Models	11
3.2.1. Car-Following Models	11
3.2.2. Lane-Change Models	12
3.3. Probabilistic Models	12
3.3.1. Hidden Markov Models	13
3.3.2. Dynamic Bayesian Networks	13
3.3.3. Gaussian-Based Probabilistic Models	13
3.4. Learning-Based Models	14
3.4.1. Classical Machine Learning Models	14
3.4.2. Recurrent Neural Network-Based Models	14

Contents

3.4.3.	Graph Neural Networks	15
3.4.4.	Transformers	15
3.4.5.	Conditional Variational Autoencoders	16
3.4.6.	Diffusion Models	17
3.4.7.	Other Learning-Based Behavior Modeling Approaches	17
3.5.	Frameworks for Autonomous Decision Making	18
3.5.1.	Game-Theoretic Models for Mixed-Traffic Interactions	18
3.5.2.	Partially Observable Markov Decision Process	18
3.5.3.	Risk Modeling in Autonomous Decision Making	18
3.6.	Comparative Assessment of Approaches for HDV Behavior Modeling and Prediction	19
4.	Methodology	21
4.1.	Proposed Framework	21
4.2.	Dataset	22
4.3.	Data Preparation and Preprocessing	23
4.3.1.	Preprocessing Pipeline	23
4.3.2.	Data Analysis for Feature Design	24
4.3.3.	Feature Engineering	27
4.3.4.	Window-Based Representation	28
4.4.	Proposed DBN Model	28
4.4.1.	Latent State Structure	29
4.4.2.	Transition Model	30
4.4.3.	Emission Model Variants	30
4.4.4.	Modeling Assumptions	31
4.5.	Model Training	32
4.5.1.	Feature Scaling	32
4.5.2.	Parameter Initialization	33
4.5.3.	EM-Based Training	33
4.6.	Inference and Evaluation	34
4.6.1.	Generative Evaluation Metrics	36
4.6.2.	Predictive Evaluation Metrics	39
4.6.3.	Posterior-Based Interpretation of Individual Predictions	40
4.6.4.	Computational Efficiency	40
4.7.	Semantic Interpretation of Latent States	41
5.	Conclusions	42
	Bibliography	43
A.	Direction-Wise Sign Convention in Raw Kinematics	49
B.	Class-Wise Distribution Plots	51

C. Complete Window-Level Observation Features	55
D. Emission Model Derivations	58
D.0.1. EM Objective	58
D.1. Hierarchical Emission Model	61
D.1.1. Parameter Updates	62
D.2. Product-of-Experts Emission Model	63
D.2.1. Implication for the M-Step	65
D.2.2. Gradient Form	66

List of Figures

2.1. Example of a Bayesian Network (BN).	5
2.2. Two-slice representation of a Dynamic Bayesian Network (DBN).	6
4.1. Overview of the proposed framework	22
4.2. Example traffic scene from a selected highD subset	23
4.3. Global distributions of the main kinematic variables.	24
4.4. Lane-change sparsity analysis in the highD data.	25
4.5. Global trajectory-level variability distributions of the main kinematic variables.	26
4.6. Selected neighbors considered in the observation	27
4.7. Illustration of the sliding-window representation used in this work. . . .	29
4.8. Factorized latent structure of the proposed DBN.	29
4.9. Emission model variants considered in the proposed DBN.	31
4.10. Causal online prediction and evaluation procedure.	36
A.1. Direction-wise sign difference in raw longitudinal velocity.	49
A.2. Class-wise raw longitudinal velocity distributions.	50
B.1. Class-wise frame-level distributions of the main kinematic variables. . .	51
B.2. Class-wise trajectory-level variability distributions of the main kinematic variables.	53

List of Tables

4.1. Frame-level feature groups used in the preprocessing pipeline	28
C.1. Window-level ego-motion features.	55
C.2. Window-level lane-change and lane-geometry features.	55
C.3. Window-level front-risk features.	56
C.4. Window-level features related to the front neighbor.	56
C.5. Window-level features related to the side neighbors.	56
C.6. Window-level features related to the adjacent front neighbors.	57

List of Abbreviations

AV Autonomous Vehicle

HDV Human-Driven Vehicle

DBN Dynamic Bayesian Network

BN Bayesian Network

DAG Directed Acyclic Graph

MAP Maximum A Posteriori

AI Artificial Intelligence

THW Time Headway

TTC Time to Collision

KI Künstliche Intelligenz

CPU Central Processing Unit

V2X Vehicle-to-Everything

RV Random Variable

List of Abbreviations

CPD Conditional Probability Distribution

HMM Hidden Markov Model

EM Expectation-Maximization

MLE Maximum Likelihood Estimation

ADAS Advanced Driver Assistance Systems

IDM Intelligent Driver Model

FVDM Full Velocity Difference Model

OVM Optimal Velocity Model

GMM Gaussian Mixture Model

GMR Gaussian Mixture Regression

GPR Gaussian Process Regression

MOBIL Minimizing Overall Braking Induced by Lane Changes

DL Deep Learning

SVM Support Vector Machine

kNN k-Nearest Neighbor

RNN Recurrent Neural Network

List of Abbreviations

LSTM Long Short-Term Memory

NN Neural Network

GNN Graph Neural Network

T-GNN Temporal Graph Neural Network

GCN Graph Convolutional Network

MTR Motion Transformer

HiVT Hierarchical Vector Transformer

PCA Principal Component Analysis

CVAE Conditional Variational Autoencoder

VAE Variational Autoencoder

PECNet Predicted Endpoint Conditioned Network

IRL Inverse Reinforcement Learning

MaxEnt IRL Maximum Entropy Inverse Reinforcement Learning

POMDP Partially Observable Markov Decision Process

PoE Product of Experts

MAP Maximum a Posteriori

List of Abbreviations

BIC Bayesian Information Criterion

TTE Time To Event

Glossary

Autonomous Vehicle (AV) A vehicle that can drive by itself without human control. It uses sensors and artificial intelligence to understand the surrounding environment and make driving decisions.

Belief State The probability distribution over the latent states given the observed data at a specific time step.

Confusion Matrix A matrix showing the performance of a classification model, across the true and predicted classes.

Directed Acyclic Graph (DAG) A graph made up of nodes and directed edges between them, where the edges do not form any closed loops.

DBN A probabilistic graphical model that uses a directed acyclic graph to represent time-series data, where each node represents a random variable and the edges show how these variables depend on each other within and across time steps.

Emission Model The probability distribution that describes how observations are generated from latent states.

Entropy A measure of uncertainty in a probability distribution. It also quantifies the amount of information needed to describe the distribution.

F1 Score The harmonic mean of precision and recall.

Human-Driven Vehicle (HDV) A vehicle that is controlled by a human driver who performs all driving tasks such as steering, accelerating, and braking.

Inference The process of estimating the probability distribution of latent states or unknown variables from observed data.

Latent State A variable that is not directly observed but is inferred from data.

Likelihood The probability of the observed data under a given model.

Maximum a Posteriori (MAP) The estimate of the most likely latent state, given the observed data. It is the state that maximizes the posterior distribution.

Mixed Traffic A traffic situation where both autonomous vehicles and human-driven vehicles operate together.

Glossary

Observation The data that is directly measured from the system, represented as feature vectors.

Posterior The probability distribution over the latent states after observing the data.

Precision The ratio of true positive predictions to the total predicted positives. It indicates the accuracy of positive predictions.

Prior The probability distribution over the latent states before observing any data.

Recall The ratio of true positive predictions to the total actual positives. It indicates the completeness of positive predictions.

Random Variable (RV) A variable that can take on different values, each with an associated probability.

State Transition Probability of transitioning from one latent state to another.

Time Headway (THW) The time needed for a vehicle to reach the current position of the vehicle ahead if it maintains its current velocity.

Time to Collision (TTC) The time needed for a vehicle to collide with the vehicle ahead if both of them maintain their current velocity.

Trajectory The time-ordered sequence describing the movement of a vehicle over time.

Kurzfassung

Künstliche Intelligenz (KI) verändert autonome Systeme durch bedeutende Fortschritte in den Bereichen Sensorik, Kommunikation und Rechenleistung. Obwohl vollständig autonome Fahrzeuge (Autonomous Vehicles (AVs)) voraussichtlich immer häufiger werden, werden sie auf absehbare Zeit gemeinsam mit von Menschen gesteuerten Fahrzeugen (Human-Driven Vehicles (HDVs)) am Straßenverkehr teilnehmen. Um in diesem gemischten Verkehrsumfeld sicher zu operieren, müssen autonome Systeme in der Lage sein, menschliche Fahrhandlungen zu modellieren und vorherzusagen.

Diese Arbeit adressiert dieses wichtige vorgelagerte Problem durch die Entwicklung eines transparenten, probabilistischen Rahmens zur Vorhersage hochrangiger menschlicher Fahrhandlungen unter Verwendung dynamischer Bayes'scher Netze (Dynamic Bayesian Networks (DBNs)). Ziel dieser Forschung ist die Entwicklung eines Modells, das zukünftige Fahreraktionen abschätzen kann und gleichzeitig Unsicherheiten in einer interpretierbaren Form darstellt. Ein solches Modell kann innerhalb eines AV-Systems als Belief-Modell dienen und Informationen für nachgelagerte Planungs- und sicherheitsrelevante Module bereitstellen.

Die Studie verwendet den naturalistischen Autobahnfahrdatensatz highD, um zeitliche Muster menschlichen Fahrverhaltens zu erlernen. Fahrzeugtrajektorien werden in sequenzielle, fensterbasierte Beobachtungen umgewandelt, die relevante Bewegungs- und Interaktionsinformationen über die Zeit erfassen. Diese Beobachtungen fassen zusammen, wie sich ein Fahrzeug und der umgebende Verkehr über kurze Zeitintervalle entwickeln. Anschließend wird ein DBN trainiert, um die zeitliche Entwicklung des Fahrerverhaltens zu modellieren und Aktionswahrscheinlichkeiten aus eingehenden Beobachtungen abzuleiten. Das vorgeschlagene DBN verwendet eine hierarchische latente Struktur, in der Aktionen von Fahrregimen abhängig sind. Genauer gesagt repräsentiert das Modell zwei latente Regime, ein Regime mit geringer Interaktion und ein Regime mit hoher Interaktion, und sagt Aktionszustände innerhalb jedes Regimes vorher. Dadurch kann das Modell sowohl das vorhergesagte Fahrmanöver als auch den Interaktionskontext darstellen, in dem es auftritt.

Eine zentrale Stärke dieses Ansatzes liegt in seiner transparenten probabilistischen White-Box-Struktur. Im Gegensatz zu vielen Black-Box-Lernmethoden erzeugt das DBN seine Vorhersagen über explizite probabilistische Beziehungen, die mithilfe der Wahrscheinlichkeitstheorie interpretiert werden können. Dies ist insbesondere für sicherheitsrelevante Anwendungen von Bedeutung, bei denen Interpretierbarkeit und die Darstellung von Unsicherheiten wichtig sind. Das Modell unterstützt daher nicht nur die Vorhersageleistung, sondern auch die Erklärbarkeit, was bei der Betrachtung solcher

Systeme für sicherheitskritische Verkehrsanwendungen von Vorteil ist. Die Studie leistet einen Beitrag zur Forschung im Bereich des autonomen Fahrens, indem sie Aktionsvorhersage, zeitliche probabilistische Schlussfolgerung und Interpretierbarkeit in einem einheitlichen Rahmen kombiniert.

Die Ergebnisse zeigen, dass das Modell in der Lage ist, aussagekräftige Verhaltensmuster aus naturalistischen Verkehrsdaten zu lernen und vielversprechende Manöर्वorhersagen über einen zukünftigen Zeithorizont zu generieren. In der aktuellen Evaluation erreichte das Modell eine exakte Next-Step-Genauigkeit von 46,4 %, eine Makropräzision von 44,3 % sowie eine Trefferquote von 68,0 % über einen Vorhersagehorizont von zwei Sekunden. Eine Vorhersage zum Zeitpunkt t wird dabei als Treffer gezählt, wenn das vorhergesagte Manöver innerhalb der nächsten fünf Fensterschritte auftritt, was etwa zwei Sekunden entspricht. Die durchschnittliche Time-to-Hit betrug 0,50 s und der Median 0,40 s, was zeigt, dass viele korrekte Vorhersagen früh innerhalb des Vorhersagehorizonts auftreten. Diese Ergebnisse deuten darauf hin, dass das Modell zukünftige menschliche Fahreraktionen innerhalb eines praktisch nutzbaren kurzfristigen Zeitintervalls antizipieren kann, anstatt nur einen einzelnen exakten zukünftigen Zeitpunkt vorherzusagen. Implementierungs-Benchmarks zeigen zudem geringe Online-Inferenzkosten mit einer mittleren CPU-Latenz von 0,164 ms pro Fahrzeug für Belief-Update und nächste Aktionsvorhersage.

Insgesamt zeigen die Ergebnisse, dass DBNs ein vielversprechender Ansatz für die probabilistische Vorhersage menschlicher Fahreraktionen sind. Die Studie demonstriert, dass hochrangige menschliche Fahrmanöver mithilfe naturalistischer Verkehrsdaten in einer interpretierbaren und zeitlich strukturierten Weise modelliert werden können. Dies stellt einen wichtigen Schritt hin zu einem besseren Verständnis menschlichen Fahrverhaltens im gemischten Verkehr dar und kann AV-Systeme dabei unterstützen, das Verhalten umliegender menschlicher Fahrer besser einzuschätzen.

Stichwörter: Vorhersage menschlichen Fahrverhaltens, Autonomes Fahren, Von Menschen gesteuerte Fahrzeuge, Dynamische Bayes'sche Netze, Manöर्वorhersage, Probabilistische Modellierung, Gemischte Verkehrsumgebungen

Abstract

Artificial Intelligence (AI) is transforming autonomous systems through major advances in sensing, communication, and computation. Although fully Autonomous Vehicles (AVs) are expected to become more common, they will share the road with Human-Driven Vehicles (HDVs) for the foreseeable future. To operate safely in this mixed-traffic setting, autonomous systems must be able to model human driver behavior and predict future maneuvers.

This work addresses this important upstream problem by developing a transparent, probabilistic framework for predicting high-level human driving behavior using Dynamic Bayesian Networks (DBNs). The purpose of this research is to develop a model that can estimate likely future maneuvers while also representing uncertainty in an interpretable form. Such a model can act as a belief model inside an AV framework and provide information for downstream planning and safety-related modules.

The study uses the highD naturalistic highway driving dataset to learn temporal patterns of human driving behavior. Vehicle trajectory data are converted into sequential window-based observations that capture relevant motion and interaction information over time. These observations summarize how the vehicle and its surrounding traffic evolve over short time intervals. A DBN is then trained to model the temporal evolution of driver behavior and to infer action probabilities from incoming observations. The proposed DBN uses a hierarchical latent structure in which actions are conditioned on driving regimes. More specifically, the model represents two latent regimes, low-interaction and high-interaction, and predicts discrete action states within each regime. This allows the framework to represent both the predicted maneuver and the interaction context in which it occurs.

A key strength of this approach is that it provides a transparent, white-box probabilistic structure. Unlike many black-box learning methods, the DBN makes predictions through explicit probabilistic relationships that can be interpreted using probability theory. This is valuable in safety-relevant applications, where interpretability and uncertainty representation are important. The model therefore supports not only prediction, but also explainability, which is useful when such systems are considered for use in safety-critical transport settings. The study contributes to science and engineering by combining maneuver prediction, temporal probabilistic reasoning, and interpretability in a single framework for autonomous driving applications.

The results show that the model can learn meaningful behavioral patterns from naturalistic traffic data and generate promising maneuver predictions over a future horizon. In the current evaluation, the model achieved 46.4% exact next-step accuracy, 44.3% macro

Abstract

precision, and a 68.0% hit rate over a 2-second prediction horizon. Here, a prediction made at time t is counted as a hit if the predicted maneuver appears at any point within the next 5 window steps, corresponding to about 2 seconds. The mean time-to-hit was 0.50 s, and the median time-to-hit was 0.40 s, showing that many correct predictions occur early within the horizon. These results suggest that the model can anticipate future human driver maneuvers within a practically useful short-term interval rather than only at one exact future instant. Implementation benchmarks also indicate low online inference cost, with a mean single-vehicle Central Processing Unit (CPU) latency of 0.164 ms for belief update and next-maneuver prediction.

Overall, the findings indicate that DBNs are a promising framework for probabilistic human driver maneuver prediction. The study shows that high-level human driving maneuvers can be modeled in an interpretable and temporally structured way using naturalistic traffic data. This provides a useful step toward better behavior understanding in mixed traffic and may support AV systems that need to reason about nearby human drivers.

Keywords: Human Driver Behavior Modeling, Autonomous Driving, Human-Driven Vehicles, Dynamic Bayesian Networks, Maneuver Prediction, Probabilistic Modeling, Mixed Traffic Environments

1. Introduction

Automation is influencing almost every part of daily life, and transportation is one of the areas changing most rapidly. Autonomous Vehicles (AVs) are at the forefront of this change, revolutionizing the transportation sector. Their development has accelerated because of recent progress in Artificial Intelligence (AI), sensing systems, computing, and communication technologies. These innovations could enhance traffic efficiency, increase travel comfort, improve road safety, and, most importantly, reshape future mobility.

However, fully autonomous traffic systems do not yet exist anywhere in the world. AVs will have to operate in mixed traffic together with Human-Driven Vehicles (HDVs). This creates important challenges because different road users provide different types and amounts of information. While AVs can rely on rich on-board sensing and may also use Vehicle-to-Everything (V2X) communication [1], the intentions of human drivers are not directly observable and must be inferred from their behavior in traffic [2]. Consequently, their intentions and decisions cannot be measured directly, but instead have to be estimated from observable cues such as vehicle motion, lane changes, relative positions, and interactions with nearby vehicles.

For this reason, human driver behavior prediction is an important problem in autonomous driving. If an AV can estimate what nearby human drivers are likely to do next, it can make safer and better-informed decisions. Anticipating likely braking, lane-changing, or car-following behavior can support downstream tasks such as risk assessment, trajectory prediction, and motion planning [2], [3], [4].

Over the last two decades, researchers have developed three broad families of behavioral models for road users: rule-based models, probabilistic models, and learning-based methods [3], [5], [6]. In parallel, several game-theoretic decision-making paradigms for AVs have emerged [7]. Rule-based models are simple and easy to understand, but they are often too rigid to capture the variability of real human behavior. Learning-based methods can learn complex patterns from large datasets, but many of them are difficult to interpret and are considered as black boxes. Probabilistic models provide a useful middle ground because they can explicitly represent uncertainty, capture temporal dependencies, and still remain interpretable.

This thesis focuses on probabilistic modeling of high-level human driving behavior in highway traffic. Rather than building the full control or planning stack of an AV, this work addresses an important upstream problem: predicting the likely future maneuvers of surrounding human drivers. The goal is to develop a model that can estimate likely future maneuvers while also representing uncertainty in a clear and interpretable way. In this thesis, future maneuvers are represented internally as discrete action states.

1. Introduction

To achieve this, the thesis uses a Dynamic Bayesian Network (DBN) to model the temporal relationship between observable driving features and hidden behavioral states. Since driver intentions are not directly observable, the model represents them through latent variables and infers them probabilistically from trajectory-based observations [8]. In particular, the DBN is designed to capture latent factors such as driving context and action dynamics, and to produce probabilistic predictions of future high-level maneuvers. The model is trained and evaluated using the highD dataset, a naturalistic highway driving dataset containing detailed vehicle trajectory recordings [9]. From these trajectories, relevant features are extracted to describe motion, surrounding traffic context, and interaction behavior. These features serve as observations for the DBN.

The thesis studies whether such a probabilistic framework can provide accurate, interpretable, and computationally efficient short-horizon predictions of future human driving maneuvers. In particular, it examines the suitability of the dataset, the structure of the DBN, the usefulness of the selected observation features, the quality of future maneuver prediction, the interpretability of the learned states, and the runtime performance of the model for online use.

2. Theoretical Background

This chapter presents the theoretical background required for the methods used later in this thesis. Section 2.1 introduces the main concepts from probability theory. Section 2.2 then describes probabilistic graphical models, including Bayesian Networks (BNs) and DBNs. Section 2.3 presents inference in chain-structured DBNs. Section 2.4 summarizes parameter learning using the Expectation-Maximization (EM) algorithm.

Notation convention Throughout this thesis, capital letters denote Random Variables (RVs), while lowercase letters denote specific realizations of those variables. Bold lowercase symbols denote vectors, bold uppercase symbols denote matrices, and calligraphic uppercase symbols denote third-order (or higher-order) tensors.

2.1. Probability Theory

Probability theory provides a mathematical framework for representing and reasoning about uncertainty [10]. Instead of assigning a single deterministic outcome to an uncertain event, probabilistic models provide a probability distribution over possible outcomes. Each uncertain quantity is represented as a RV [10].

Complex systems require modeling interactions between many RVs. When several RVs are considered together, their behavior can be described using a joint probability distribution [10]. For two RVs X and Y , the joint probability is written as $P(X, Y)$ and represents the probability that both variables take specific values at the same time. This idea can be extended to any number of variables.

Dependencies between RVs are described using conditional probability [10]. The conditional probability of X given Y is defined as

$$P(X | Y) = \frac{P(X, Y)}{P(Y)},$$

provided that $P(Y) > 0$. Conditional probability describes how the probability of one RV changes when information about another RV is known.

Another important concept is marginal probability, which gives the probability of a subset of RVs irrespective of the values of the remaining variables [10]. For example, if the joint distribution $P(X, Y)$ is known, the marginal probability of Y can be obtained by summing over all possible values of X :

$$P(Y) = \sum_X P(X, Y)$$

2. Theoretical Background

In addition to describing dependencies between RVs, it is also important to identify cases in which variables do not influence each other. Two RVs are said to be independent if the value of one variable does not affect the probability of the other. For independent variables X and Y , the joint probability can be written as:

$$P(X, Y) = P(X)P(Y)$$

In many practical problems, variables are not completely independent but may become independent when conditioned on another variable. This property is known as conditional independence. Two variables X and Y are conditionally independent given a third variable Z if the joint conditional distribution factorizes as:

$$P(X, Y | Z) = P(X | Z)P(Y | Z)$$

Conditional independence is a key concept in probabilistic graphical models because it allows complex joint probability distributions to be represented using simpler conditional relationships between variables [11].

2.1.1. Bayes' Theorem

A fundamental principle in probabilistic reasoning is Bayes' theorem, which allows beliefs about uncertain events to be updated when new observations become available [10]. It relates conditional and marginal probabilities and is expressed as:

$$P(X | Y) = \frac{P(Y | X)P(X)}{P(Y)} \quad (2.1)$$

Here, $P(X)$ is called the prior probability, $P(Y | X)$ is called the likelihood, $P(Y)$ is called the evidence or marginal likelihood, and $P(X | Y)$ is called the posterior probability. Bayes' theorem therefore provides a systematic way to update probabilistic beliefs when new observations become available. This principle forms the basis for probabilistic inference in many models, including DBNs.

2.2. Probabilistic Graphical Models

Probabilistic graphical models provide an efficient way to represent complex joint probability distributions over many RVs [10], [11]. As the number of variables increases, the full joint distribution becomes harder to model because possible dependencies also grows. Probabilistic graphical models address this problem by representing RVs and their probabilistic relationships using a graph. In this representation, nodes correspond to RVs, while edges describe dependencies between them. The graph structure also encodes conditional independence assumptions. These assumptions allow the joint probability distribution to be factorized into simpler Conditional Probability Distributions (CPDs).

There are two main types of probabilistic graphical models:

2. Theoretical Background

1. **Directed graphical models**, where edges have a direction and represent conditional dependencies between variables.
2. **Undirected graphical models**, where edges do not have a direction and represent symmetric relationships between variables.

One important class of probabilistic graphical models is the BN. Its temporal extension is the DBN, which is particularly suitable for modeling sequential processes over time.

2.2.1. Bayesian Networks

A BN is a directed probabilistic graphical model that represents a set of RVs and their conditional dependencies using a Directed Acyclic Graph (DAG) [11]. Each variable in the network has an associated CPD, which describes how that variable depends on its parent variables. If a variable has no parents in the graph, its distribution is given by a prior probability distribution.

In general, the joint probability distribution of all variables in a BN can be factorized according to the graph structure. For a set of variables $X_1, X_2, \dots, X_n$, the joint distribution can be written as:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Pa}(X_i)) \quad (2.2)$$

where $\text{Pa}(X_i)$ denotes the set of parent variables of X_i in the graph. This factorization reflects the conditional independence assumption embedded in the network structure.

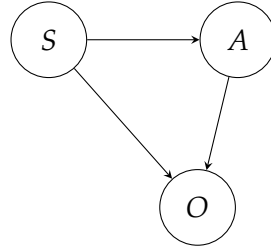


Figure 2.1.: Example of a BN.

For the example network shown in Figure 2.1, the joint distribution can be written as:

$$P(S, A, O) = P(S)P(A \mid S)P(O \mid S, A)$$

Standard BNs describe relationships between variables within a single time step. However, in many real-world problems, the system evolves over time. In such cases, a temporal extension of BNs, called a DBN, is used.

2.2.2. Dynamic Bayesian Networks

In a DBN, the system is represented as a sequence of time-indexed RVs that describe how the system evolves over time. A common representation of a DBN uses a two-slice temporal model that captures dependencies both within a single time step and across consecutive time steps. This representation typically relies on the first-order Markov assumption, which states that the current state depends only on the previous state and not on earlier states. Unlike an Hidden Markov Model (HMM), which can be viewed as a special case of a DBN with a single latent state per time step, a general DBN may contain multiple latent states with structured dependencies between them.

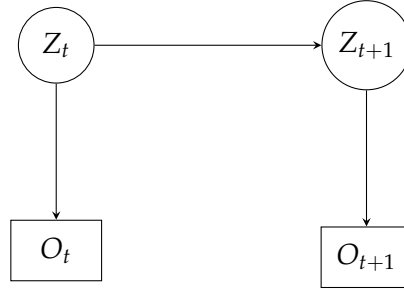


Figure 2.2.: Two-slice representation of a DBN.

In the example shown in Figure 2.2, let Z_t denote the latent state at time step t and O_t denote the corresponding observation. Under the Markov assumption, the joint probability distribution over a sequence of states and observations can be written as:

$$P(Z_{1:T}, O_{1:T}) = P(Z_1) \prod_{t=2}^T P(Z_t | Z_{t-1}) \prod_{t=1}^T P(O_t | Z_t) \quad (2.3)$$

In this formulation, $P(Z_t | Z_{t-1})$ represents the state transition model, which captures how the latent state evolves over time, while $P(O_t | Z_t)$ represents the emission model that relates the hidden state to the observed data.

2.3. Inference in Chain-Structured DBNs

Inference in probabilistic graphical models refers to estimating the probability distribution of unknown variables conditioned on observed evidence. In temporal models like the DBN shown in Figure 2.2, inference involves estimating the distribution of latent states from a sequence of observations, since only the observations $O_{1:T}$ are available while the state sequence $Z_{1:T}$ remains hidden [10], [11].

In temporal probabilistic models, three common inference tasks arise [12]:

1. **Filtering:** Estimating the current latent state using observations up to the current time step, $P(Z_t | O_{1:t})$.

2. Theoretical Background

2. **Smoothing:** Estimating the latent state using the full observation sequence, $P(Z_t | O_{1:T})$.
3. **Prediction:** Estimating future latent states from the observations available up to the current time, $P(Z_{t+k} | O_{1:t})$.

For the chain-structured DBN shown in Figure 2.2, the latent state at time t depends only on the previous state Z_{t-1} , and the observation O_t depends only on the current state Z_t . This structure corresponds to a first-order Markov model with conditionally independent observations, which is equivalent to the structure of a HMM. Therefore, exact inference can be performed using the forward-backward algorithm [13]. This algorithm is widely used in HMMs, which are a special case of DBNs.

2.3.1. Forward–Backward Algorithm

The forward-backward algorithm computes posterior probabilities of latent states by combining information from past and future observations [13]. More specifically, it estimates the posterior distribution of the hidden state at each time step given the complete observation sequence.

Let the observation sequence be $O_{1:T} = \{O_1, O_2, \dots, O_T\}$ and the corresponding latent states be $Z_{1:T}$. The main objective is to compute the posterior distribution, $P(Z_t | O_{1:T})$, which is called the smoothing distribution.

The forward-backward algorithm performs this computation using two recursive quantities: the forward messages and the backward messages.

Forward Pass

The forward variable $\alpha_t(z_t)$ represents the joint probability of observing the partial sequence up to time t and being in state z_t :

$$\alpha_t(z_t) = P(O_{1:t}, Z_t = z_t) \quad (2.4)$$

It can be computed recursively as:

$$\alpha_t(z_t) = P(O_t | Z_t = z_t) \sum_{z_{t-1}} P(Z_t = z_t | Z_{t-1} = z_{t-1}) \alpha_{t-1}(z_{t-1}) \quad (2.5)$$

with initialization:

$$\alpha_1(z_1) = P(Z_1 = z_1) P(O_1 | Z_1 = z_1)$$

Here, $P(Z_t = z_t | Z_{t-1} = z_{t-1})$ is the transition probability, $P(O_t | Z_t = z_t)$ is the observation likelihood, and $P(Z_1 = z_1)$ is the initial state distribution.

2. Theoretical Background

Backward Pass

The backward variable $\beta_t(z_t)$ represents the probability of observing the remaining sequence from time $t + 1$ to T given the current state:

$$\beta_t(z_t) = P(O_{t+1:T} \mid Z_t = z_t) \quad (2.6)$$

It is computed recursively in the reverse direction as:

$$\beta_t(z_t) = \sum_{z_{t+1}} P(Z_{t+1} = z_{t+1} \mid Z_t = z_t) P(O_{t+1} \mid Z_{t+1} = z_{t+1}) \beta_{t+1}(z_{t+1}) \quad (2.7)$$

with initialization:

$$\beta_T(z_T) = 1$$

Posterior State Estimation

Once the forward and backward messages have been computed, the posterior probability of the hidden state at time step t can be calculated. This quantity is commonly denoted as $\gamma_t(z_t)$ and represents the probability that the system is in state z_t at time t given the full observation sequence:

$$\gamma_t(z_t) = P(Z_t = z_t \mid O_{1:T})$$

Using the forward and backward variables, this can be written as:

$$\gamma_t(z_t) = \frac{\alpha_t(z_t) \beta_t(z_t)}{P(O_{1:T})} \quad (2.8)$$

where $P(O_{1:T})$ is the likelihood of the full observation sequence. Using the forward and backward variables, for any time step t , it can be written as:

$$P(O_{1:T}) = \sum_{z_t} \alpha_t(z_t) \beta_t(z_t)$$

The forward-backward algorithm therefore combines information from earlier observations through the forward pass and from later observations through the backward pass.

Posterior Transition Probability

In addition to the posterior probability of individual states, it is also useful to compute the joint posterior probability of two consecutive states. This quantity gives the posterior probability of a transition from state z_t to state z_{t+1} given the full observation sequence:

$$\xi_t(z_t, z_{t+1}) = P(Z_t = z_t, Z_{t+1} = z_{t+1} \mid O_{1:T})$$

2. Theoretical Background

Using the forward and backward variables, this can be written as:

$$\xi_t(z_t, z_{t+1}) = \frac{\alpha_t(z_t) P(Z_{t+1} = z_{t+1} \mid Z_t = z_t) P(O_{t+1} \mid Z_{t+1} = z_{t+1}) \beta_{t+1}(z_{t+1})}{P(O_{1:T})} \quad (2.9)$$

This posterior transition probability is useful in parameter learning, especially in EM-based training methods [13].

2.4. Parameter Learning in Latent-State Models

A probabilistic model contains several unknown parameters that must be learned from data. For a chain-structured temporal model such as the one shown in Figure 2.2, these parameters can be collectively written as:

$$\theta = \{\pi, A, \Theta\}$$

where π denotes the initial-state probability vector, A is the state transition matrix, and Θ denotes the set of emission parameters.

If both the observation sequence $O_{1:T}$ and the latent state sequence $Z_{1:T}$ were known, parameter learning could be performed by maximizing the complete-data likelihood:

$$\theta^* = \arg \max_{\theta} P(O_{1:T}, Z_{1:T} \mid \theta) \quad (2.10)$$

However, in latent-variable models the state sequence $Z_{1:T}$ is not observed. Therefore, learning must be based only on the observed data, which gives the Maximum Likelihood Estimation (MLE) objective:

$$\theta^* = \arg \max_{\theta} P(O_{1:T} \mid \theta) \quad (2.11)$$

Since the latent state sequence is unknown, the likelihood of the observations is obtained by marginalizing over all possible latent state sequences:

$$P(O_{1:T} \mid \theta) = \sum_{Z_{1:T}} P(O_{1:T}, Z_{1:T} \mid \theta) \quad (2.12)$$

Since this summation involves many possible latent state sequences, direct maximization of the observed-data likelihood in (2.11) is difficult. To address this problem, the EM algorithm is commonly used [12]. For HMMs, this learning procedure is known as the Baum–Welch algorithm [13].

2.4.1. Expectation–Maximization Algorithm

The EM algorithm is an iterative method for parameter estimation in probabilistic models with latent variables. It addresses the difficulty of maximizing the observed-data likelihood by alternating between estimating the latent states and updating the model parameters.

2. Theoretical Background

Expectation Step

In the expectation step (E-step), the current parameter estimate is used to compute the posterior distribution of the latent states given the observations. For the chain-structured model considered here, this is done using the forward-backward algorithm described by (2.5) and (2.7). This step provides the posterior state probabilities and the posterior transition probabilities:

$$\gamma_t(i) = P(Z_t = i \mid O_{1:T})$$

$$\xi_t(i, j) = P(Z_t = i, Z_{t+1} = j \mid O_{1:T})$$

As shown in (2.8) and (2.9), these posterior quantities represent the probabilities of state occupancy and state transitions conditioned on the full observation sequence.

Maximization Step

In the maximization step (M-step), the model parameters are updated using the posterior quantities computed in the E-step.

For discrete-state chain-structured models, the initial state distribution is updated as:

$$\pi_i = \gamma_1(i) \tag{2.13}$$

The transition probabilities can be re-estimated as:

$$A_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)} \tag{2.14}$$

The parameters of the emission model are updated using the observations weighted by the posterior state probabilities. The exact form of this update depends on the chosen emission model.

The E-step and M-step are repeated until convergence. Under standard EM, each iteration does not decrease the observed-data likelihood, and the algorithm converges to a local optimum or stationary point [12].

3. State of the Art

3.1. Introduction

Existing approaches for modeling and predicting human driver behavior can be broadly grouped into rule-based models, probabilistic models, and learning-based models. These families differ in how they represent driving behavior, uncertainty, and interactions with the surrounding traffic environment.

This chapter reviews the main state-of-the-art approaches for human driver behavior modeling and maneuver prediction, with particular attention to their suitability for uncertainty representation and interpretability. Since human driver maneuver prediction is intended to support autonomous driving systems, the chapter also briefly discusses how such predictions are used in downstream decision making, including interaction-aware planning and risk-aware reasoning.

3.2. Rule-Based Models

Rule-based models characterize driving behavior through explicit mathematical or logical rules. These models relate observable variables such as spacing, relative velocity, and desired speed to longitudinal acceleration or lane-change decisions. Their main advantages are interpretability and low computational cost [14]. For this reason, they are widely used in traffic simulation, Advanced Driver Assistance Systems (ADAS), and baseline behavior modeling. However, as they rely on hand-crafted assumptions about driver decision-making, they often represent behavior in a deterministic or simplified manner.

3.2.1. Car-Following Models

Intelligent Driver Model

The Intelligent Driver Model (IDM) is one of the most widely used car-following models [14]. It expresses a vehicle's acceleration as a continuous function of its current velocity, desired velocity, spacing to the leading vehicle, and relative velocity [15]. The formulation also includes parameters related to desired time headway and comfortable braking, which make the model suitable for safety-oriented traffic simulation. Because of its simple structure and physically meaningful parameters, the IDM remains a common reference model in intelligent driving research.

Gipps' Model

Gipps' model is one of the earliest rule-based car-following models. It is based on the idea that a driver chooses a speed that is both desirable and safe with respect to the preceding vehicle [16]. This safety-oriented formulation allows the model to capture conservative driving behavior and ensures collision-free motion under typical traffic conditions.

Optimal Velocity Model and Full Velocity Difference Model

Another classical approach is the Optimal Velocity Model (OVM), in which the driver's acceleration depends on the difference between the current velocity and an optimal velocity determined by the spacing to the leading vehicle [17]. This model captures the driver's intuition to accelerate when the gap ahead increases and decelerate when it decreases. An extension of the OVM is the Full Velocity Difference Model (FVDM), where, instead of only considering the gap between the vehicles, the model also captures the velocity difference between the ego and preceding vehicles when calculating the acceleration [18]. This helps reduce the oscillations observed in the original OVM.

3.2.2. Lane-Change Models

MOBIL Model

The Minimizing Overall Braking Induced by Lane Changes (MOBIL) model is a lane-changing model built upon the IDM to describe lateral driving behavior [14], [19]. It incorporates incentive and safety criteria to evaluate the feasibility of lane changes. The incentive criterion indicates whether a lane change could be advantageous in terms of the driver's acceleration, whereas the safety criterion ensures that the lane change does not cause unnecessary deceleration for surrounding vehicles.

Despite their interpretability and efficiency, rule-based models have limited ability to represent uncertainty and inter-driver variability in complex traffic scenarios. These limitations motivate the use of probabilistic and learning-based approaches.

3.3. Probabilistic Models

Probabilistic models describe driving behavior as a stochastic process, where driving actions are mapped to probability distributions, and represent uncertainty explicitly [20]. In this framework, a driver's decision to accelerate or change lanes is modeled as a probability function that depends on the current traffic context. This approach allows for multiple possible behavioral outcomes, rather than a single deterministic response. This makes them suitable for modeling the variability and ambiguity that are inherent in human driving behavior.

3.3.1. Hidden Markov Models

The HMM is a widely used probabilistic framework for modeling time-varying stochastic processes in which the underlying system states are not directly observable but produce observable outputs [13]. HMM assumes that human driving behavior evolves through a sequence of decision states, such as lane-keeping, braking, and overtaking, that are not directly measurable [21]. These states produce observable vehicle signals like speed, acceleration, and lane position. The objective of the HMM is to infer the most probable state sequence and predict future states from the observed driving data. The main strengths of HMMs are conceptual simplicity and efficient inference. However, HMMs rely on the Markov property, which simplifies the model but limits its ability to capture long-term temporal dependencies and complex contextual interactions [22].

3.3.2. Dynamic Bayesian Networks

DBNs extend static BNs to temporal settings and model dependencies among variables across successive time steps [8], [23]. Compared with HMMs, DBNs provide a more flexible representation because the system state can be represented in factored form rather than as a single discrete variable. This allows DBNs to represent multiple interacting hidden and observed variables, which makes them suitable for modeling the joint evolution of driver state, vehicle behavior, and surrounding traffic context [24]. For instance, Młynarczyk et al. (2025) applied BNs to capture causal links between drivers' affective states and behavioral responses [25], while Talluri et al. (2025) employed DBNs to enhance safety standards in automated driving systems through causal reasoning for lane-change prediction and safety assessment [26]. By explicitly modeling dependencies between hidden cognitive states and observable signals, DBNs provide an excellent framework for intention recognition and maneuver prediction in autonomous driving systems [24].

3.3.3. Gaussian-Based Probabilistic Models

Gaussian-based probabilistic models are widely used for modeling continuous-valued driving behaviors such as acceleration or trajectory changes. A single Gaussian distribution is unimodal, whereas driver behavior in a given traffic context may admit multiple plausible future actions or trajectories [27]. To capture such variability, Gaussian Mixture Models (GMMs) represent the joint distribution of relevant variables, such as positions, velocities, headways, or steering angles, as a weighted sum of multiple Gaussian components. For the behavioral modeling of HDVs at the operational level, probabilistic regression techniques such as Gaussian Mixture Regression (GMR) and Gaussian Process Regression (GPR) are often employed to model continuous driving actions, like acceleration or steering angle, for a given traffic context [28], [29]. For example, Zhang et al. use a GMR model to capture feature correlations and temporal dynamics in car-following behavior [28], while Bethge et al. use Gaussian processes to predict human driving behavior and explicitly account for predictive uncertainty [29]. These models

provide low-level behavior representations by capturing the stochastic nature of human responses.

3.4. Learning-Based Models

Deep Learning (DL) has enabled powerful data-driven models that predict future trajectories directly from historical motion, map information, and interaction context. Unlike traditional rule-based or probabilistic frameworks that rely on explicit assumptions about driver intent or traffic interactions, these models learn patterns directly from extensive driving data. By making use of large datasets, they can capture complex, nonlinear relationships between driving context and human decisions with high accuracy.

3.4.1. Classical Machine Learning Models

Classical machine learning methods formed the earliest data-driven foundation for maneuver prediction before the advent of modern DL. Algorithms such as k-Nearest Neighbors (kNNs), Support Vector Machines (SVMs), Decision Trees, and Random Forests were widely applied to predict driver intent, classify maneuvers, or recognize driving styles [30]. These approaches rely on manually designed features rather than automatically learned representations, as used in modern DL approaches [31].

3.4.2. Recurrent Neural Network-Based Models

Recurrent Neural Networks (RNNs) form a specialized class of Neural Networks (NNs) designed to model temporal relationships in sequential data [32]. They incorporate internal states and cyclic connections that enable them to process time-dependent information, making them suitable for driver intention recognition and trajectory prediction in behavioral prediction tasks [33], [34]. However, traditional RNNs suffer from the vanishing gradient problem [35]. To address this limitation, the Long Short-Term Memory (LSTM) architecture was introduced. LSTM networks incorporate a dedicated memory cell that helps preserve contextual information over longer time horizons. Although such models perform well in vehicle prediction tasks, they still face limitations in long-horizon forecasting and in handling complex road geometries [33], [36].

Xin et al. (2019) proposed an intention-aware dual LSTM network that separately models the driver's intention and motion dynamics [33]. The first LSTM module captures the latent driving intention using past motion data and environmental context, while the second predicts the future trajectory conditioned on the inferred intention. This dual-stage design allows the model to reason about how inferred driver intentions influence subsequent maneuvers.

Hao et al. (2024) present a hybrid framework that combines a DBN for lane-change intention prediction with a driving-style estimator constructed from K-means clustering and Bayesian filtering [34]. These probabilistic cues are then integrated into an LSTM

3. State of the Art

encoder—decoder, which produces personalized future trajectories conditioned on both driver intent and style. This fusion allows the network to capture maneuver likelihoods and behavioral variability more effectively than pure LSTM models.

3.4.3. Graph Neural Networks

Modeling the relational structure among traffic participants is important in dynamic traffic scenes. Graph Neural Networks (GNNs) provide an effective framework for representing and learning such multi-agent interactions. In a GNN, each vehicle can be represented as a node, while edges encode the interactions between vehicles [37]. Recent research has explored integrating GNNs with temporal sequence models such as LSTMs and Transformers to jointly capture spatial and temporal dependencies. Abdelraouf et al. (2023) proposed a Temporal Graph Neural Network (T-GNN) that combines Graph Convolutional Networks (GCNs) with LSTM layers to personalize trajectory prediction for different driving styles [38].

However, the performance of GNNs depends strongly on the design of the interaction graph, since graph construction determines which neighboring agents and relations are modeled [37]. An overly sparse graph structure may lead to loss of critical interaction information, while overly dense connections can introduce noise and computational complexity. Moreover, most existing GNN-based trajectory prediction models are highly data-driven and require large, diverse datasets for generalization.

3.4.4. Transformers

Transformers are a recent and powerful DL architecture that are particularly effective at capturing long-range dependencies in sequential data. Unlike recurrent networks, they rely on self-attention mechanisms that directly model relationships between all elements in a sequence, regardless of their temporal distance. In addition, self-attention enables parallel processing of the entire input sequence, which improves computational efficiency [39], [40]. In the multi-head self-attention mechanism, different attention heads can focus on different aspects of the input, and the resulting representations are aggregated to predict future maneuvers or trajectories.

SceneTransformer introduced a model that unified motion prediction and planning through a scene-level attention mechanism that jointly predicts the trajectories of all agents within a traffic scene [41]. Instead of treating each vehicle as an independent entity, SceneTransformer predicts the joint future of multiple interacting agents. Motion Transformer (MTR) extended this approach by introducing a hierarchical encoder—decoder framework that explicitly models behavioral uncertainty, producing multiple possible future trajectories per agent [42]. Building upon these advances, the Hierarchical Vector Transformer (HiVT) introduces a multi-scale hierarchical attention structure designed to handle large numbers of interacting agents efficiently [43]. HiVT first applies local attention to encode trajectory and map features for each agent, then applies a global attention

3. State of the Art

layer to model interactions among all agents in the scene. Using these representations, the decoder predicts the future trajectories of all agents in a single forward pass.

However, despite their strong representation capacity, Transformer-based models generally require large datasets and careful regularization to avoid overfitting. Moreover, their interpretability remains limited compared to probabilistic graphical models or physics-based approaches.

3.4.5. Conditional Variational Autoencoders

Conditional Variational Autoencoders (CVAEs) are probabilistic deep generative models that extend the traditional Variational Autoencoder (VAE) framework by incorporating contextual information into both the encoder and decoder networks [44]. In trajectory prediction, the input observation x captures contextual information while the corresponding output y represents the future trajectory to be predicted. A latent variable z is introduced such that its conditional posterior is approximated through $q_\phi(z | x, y)$, which is learned by the encoder.

By conditioning both the encoder and decoder on the contextual variable x , a CVAE can capture complex, multimodal relationships between the observed scene and possible futures, enabling diverse yet context-consistent predictions. The latent variable z captures hidden factors, such as intent or other stochastic influences, that are not directly observable from the input context. As a result, the model can generate a range of possible future trajectories that represent different behavioral modes within the same traffic scenario.

Trajectron++ represents one of the popular CVAE-based frameworks for multimodal trajectory prediction. It combines LSTM-based temporal encoding, graph-structured interaction modeling, and probabilistic latent variable reasoning [45]. Each traffic participant is represented as a node in a dynamically evolving spatiotemporal graph, where edges capture interactions with surrounding agents. The past motion history of each agent is first processed through an LSTM network to encode temporal dependencies, after which a GNN aggregates contextual information across agents. The resulting embeddings are passed into a CVAE decoder, which generates diverse future trajectories by sampling from a learned latent space.

While Trajectron++ focuses on interaction-aware multimodality, Predicted Endpoint Conditioned Network (PECNet) follows a related goal-conditioned approach to trajectory prediction [46]. It models driver intent by predicting a distribution over potential final endpoints, which are spatial points that represent the driver’s intended goal. The model then generates complete trajectories conditioned on these sampled endpoints. This two-step design separates high-level intention from detailed motion planning.

More recent research has explored hierarchical extensions of the CVAE framework to reduce long-horizon prediction error and maintain trajectory consistency over time. Single-stage CVAE models can suffer from reduced prediction quality at longer horizons. For example, the Cascaded CVAE addresses this by refining predictions through multiple

3. State of the Art

CVAE stages, where each stage predicts short-term motion and subsequently extends or corrects it over longer horizons [47].

3.4.6. Diffusion Models

Diffusion models are generative models that learn a data distribution through a forward process that gradually adds noise to training samples and a reverse process that learns to remove this noise [48]. During inference, the model starts from random noise and iteratively denoises it into a future trajectory conditioned on the observed scene context. In trajectory prediction, this allows the model to generate multiple plausible future motions instead of a single deterministic output, which is useful because vehicle behavior is inherently uncertain and often multimodal [49], [50].

TrajDiffuse is a recent development in trajectory prediction that applies diffusion modeling. It generates accurate and diverse single-agent trajectory predictions while maintaining consistency with environmental constraints [51]. MotionDiffuser extends this concept to multi-agent interaction scenarios [52]. It employs a conditional diffusion process to generate permutation-invariant joint trajectories for multiple interacting agents and integrates a Principal Component Analysis (PCA) based dimensionality reduction technique to improve computational efficiency.

3.4.7. Other Learning-Based Behavior Modeling Approaches

Maximum Entropy Inverse Reinforcement Learning (IRL) is a learning-based behavior modeling approach that infers a latent reward function from demonstrations and models behavior as a probabilistic distribution over trajectories [53], [54]. Rather than directly mapping states to actions, IRL aims to recover the latent reward or cost function that a human driver may implicitly optimize during driving. Classical IRL assumes that the expert follows an optimal policy for an unknown reward, without accounting for variations in actual human behavior [55]. However, this can lead to ambiguity, since multiple reward functions may explain the same observed behavior. To address this, Maximum Entropy Inverse Reinforcement Learning (MaxEnt IRL) introduces the principle of maximum entropy [54]. Instead of selecting a single deterministic explanation, it learns the maximum-entropy distribution over trajectories that remains consistent with the demonstrated feature statistics. This makes the model better suited to capturing uncertainty and variability in human driving behavior. Recent work has also applied MaxEnt IRL to autonomous driving trajectory planning and interaction-aware decision making, for example in merging scenarios [56].

3.5. Frameworks for Autonomous Decision Making

3.5.1. Game-Theoretic Models for Mixed-Traffic Interactions

Level-k Reasoning Level-k models assume that different drivers operate at different reasoning levels. A level-0 driver is nonstrategic and follows a fixed, non-strategic policy like a standard car-following policy. A level-1 driver best responds to the forecasted behavior of level-0 drivers, a level-2 driver responds to level-1 behavior, and so on [57].

Stackelberg Games In Stackelberg games, AV is often modeled as the leader and the other traffic participants as followers [58]. The leader chooses its action while anticipating the followers' best responses. Correspondingly, each follower chooses a strategy that maximizes its own reward given the leader's decision. The resulting solution is commonly formulated in terms of a Stackelberg equilibrium. This framework has been applied, for example, to merging and lane-changing scenarios where a self-driving car interacts with a human driver [59]. For AVs, Stackelberg formulations are well suited because they align with the idea of the AV as a proactive but cautious decision-maker.

Bayesian Games Bayesian games model interactions where agents do not have full knowledge of each other's characteristics, such as driving style, reaction time, or capability level. Each agent maintains probabilistic beliefs over the other agent's "type" and chooses actions that maximize their expected gain under this uncertainty [60]. This framework is appealing to mixed-traffic risk assessment, where behavioral variability and environmental uncertainty play a crucial role.

3.5.2. Partially Observable Markov Decision Process

Although not a game-theoretic model, Partially Observable Markov Decision Processes (POMDPs) are often discussed alongside strategic frameworks because they explicitly reason about hidden intentions and uncertain environment states through belief updates. In a POMDPs, the AV maintains a probability distribution over latent variables, such as driver intent, and updates this belief through Bayesian filtering. Prior work has applied POMDPs to interactive driving scenarios, where accounting for uncertainty in other road users' behavior is essential for safe and anticipatory maneuver planning [61]. However, POMDP-based planners remain computationally demanding, which restricts their use in real-time autonomous driving.

3.5.3. Risk Modeling in Autonomous Decision Making

In autonomous driving, risk is often characterized using criticality indicators that estimate how close a traffic interaction is to becoming unsafe. Many existing approaches rely on surrogate safety measures derived from the relative motion between the ego vehicle and surrounding traffic. In particular, time-based indicators such as Time to Collision (TTC)

3. State of the Art

and Time Headway (THW) remain widely used, especially for collision risk estimation and longitudinal safety assessment [62], [63].

Such indicators are frequently incorporated into planning and collision-avoidance systems as safety constraints or risk terms. However, they are usually most suitable for specific interaction types, such as longitudinal following or merging, and they do not always capture complex multi-vehicle interactions well [62], [63].

To address these limitations, Nyberg et al. (2021) proposed a risk measure derived directly from a formal safety specification rather than relying only on surrogate indicators [64]. Their method computes both the probability that the AV violates a safety rule and the expected severity of that violation under uncertainty. This severity-aware violation measure is then incorporated into the planner, enabling the decision maker to select trajectories that explicitly balance risk and driving progress [64].

3.6. Comparative Assessment of Approaches for HDV Behavior Modeling and Prediction

Rule-based models are simple, interpretable, and computationally efficient. Their parameters have a direct physical meaning, and they are widely used in microscopic traffic simulation and as baselines for evaluating more complex models. However, they typically treat driving as a largely deterministic process and do not account well for the uncertainty and variability inherent in human decision-making. As a result, they are limited in their ability to capture latent intent and behavioral adaptation in complex real-world scenarios.

Probabilistic models address these limitations by treating driver behavior as a stochastic process. They can represent multiple possible maneuvers and quantify uncertainty over future actions. Within this family, DBNs offer a useful compromise between expressive power and interpretability, since they allow richer dependency structures than simpler models such as HMMs while retaining an explicit probabilistic representation of latent variables. This makes them attractive for HDV modeling, especially when uncertainty, partial observability, and belief updating are important. However, probabilistic models still require careful model design and representative data, and simpler variants may rely on restrictive conditional independence or parametric assumptions.

Learning-based models, especially deep neural architectures, currently form a major part of the state of the art in trajectory prediction on large benchmark datasets. They can capture complex nonlinear dependencies, model interactions between multiple agents, and generate multimodal futures through latent-variable or generative formulations. However, these benefits come at a cost. First, they often depend on large, diverse, annotated datasets, which may be limited for the specific mixed-traffic scenarios. Second, their internal representations are often difficult to interpret, making it harder to relate model outputs to driver intent, uncertainty sources, or downstream risk reasoning.

3. State of the Art

Given the goals of this thesis, probabilistic models, and DBNs in particular, offer the most suitable compromise. They are expressive enough to encode latent driver states and maneuver tendencies. At the same time, they remain interpretable and compatible with Bayesian belief updating and downstream risk-aware reasoning. Rule-based models remain useful as baselines and for scenario construction, whereas DL models provide an important reference for the current trajectory prediction frontier.

4. Methodology

This chapter presents the methodology used to model and predict short-horizon HDV maneuvers in highway traffic. Starting from raw vehicle trajectories in the highD dataset, the pipeline constructs a vehicle-centric representation, extracts interaction and risk-related features, summarizes them over short time windows, and uses the resulting observation sequences to train and evaluate the proposed DBN.

In highway traffic, a similar traffic situation can lead to several plausible driver responses. For example, when approaching a slower vehicle, a driver may brake and continue following or change lanes to overtake, depending on the surrounding traffic context. The resulting maneuver is therefore uncertain. To capture this uncertainty, the proposed method adopts a probabilistic model.

The goal is to predict a probability distribution over discrete high-level action classes, such as acceleration, braking, lane changes, and stable following, rather than an exact future trajectory. To achieve this, the observation space combines ego-motion variables with contextual information from surrounding vehicles and risk-related quantities.

The remainder of this chapter describes the overall framework in Section 4.1, the dataset in Section 4.2, the preprocessing and feature-engineering pipeline in Section 4.3, the proposed DBN model in Section 4.4, the training procedure in Section 4.5, the inference and evaluation procedure in Section 4.6, and the semantic interpretation of latent states in Section 4.7.

4.1. Proposed Framework

The framework converts raw highD trajectory data into temporal observations for action prediction. Its main stages are data preparation, DBN training, and inference.

The pipeline starts from frame-level trajectories and transforms them into a consistent vehicle-centric representation. Based on the ego vehicle and its local traffic environment, additional features are derived to capture interaction context, lane-related information, and risk-related properties such as THW and TTC. These steps provide a compact description of the current driving situation beyond the ego vehicle.

Action prediction depends on both the current traffic state and its recent evolution. For this reason, frame-level signals are aggregated over short sliding windows to form a window-based sequence representation. Each window becomes one observation for a DBN with two discrete latent variables: a style state and an action state. The engineered window features are linked to these latent states through the emission model, while temporal behavior is captured by the state transition model.

4. Methodology

Model parameters are estimated from the training data using an EM-based learning procedure. Each windowized observation sequence corresponds to a single vehicle trajectory. To prevent leakage across the training, validation, and test sets, the data are partitioned at the vehicle level into disjoint subsets. Specifically, the split contains 35,318 training vehicles (70%), 5,045 validation vehicles (10%), and 10,092 test vehicles (20%). The training set is used for parameter estimation, the validation set is used to monitor validation log-likelihood and to apply early stopping during EM training, and the test set is reserved for the final evaluation. After training, inference is performed on the held-out test set to estimate the posterior distribution over latent states and to predict the distribution over future maneuvers.

An overview of the complete framework is shown in Figure 4.1. The individual stages are described in detail in the following sections.

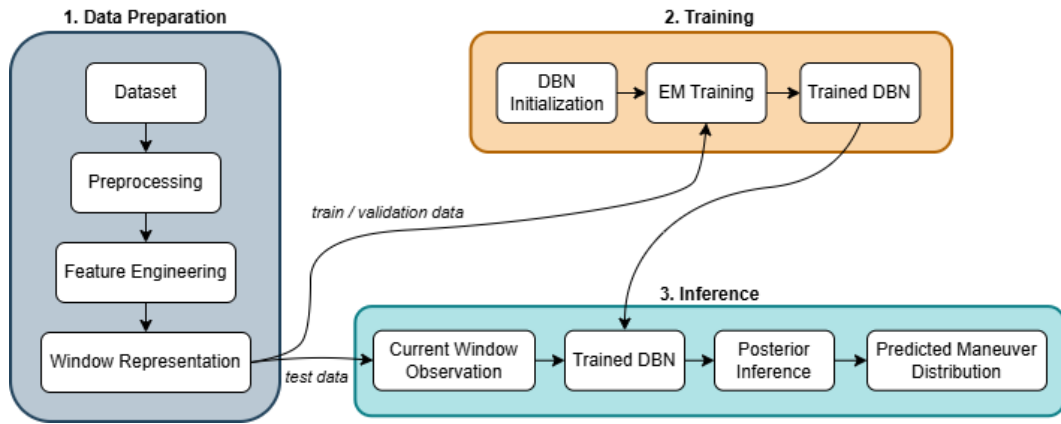


Figure 4.1.: Overview of the proposed framework

4.2. Dataset

This thesis uses the highD dataset [9], a naturalistic highway-traffic dataset recorded from drone videos on German highways. Its aerial perspective provides frame-level trajectories together with surrounding-vehicle information, making it well suited for the analysis of human driving behavior in highway scenarios.

The full highD dataset contains 60 recordings collected at several highway locations. In this work, only the first 25 recordings with three lanes in each driving direction are used. This restriction provides a more uniform road geometry and traffic setting across the selected subset, which supports more consistent behavior modeling. Figure 4.2 shows an example traffic scene from the selected subset and illustrates the multi-lane highway layout together with the surrounding-vehicle context available for each target vehicle.

4. Methodology

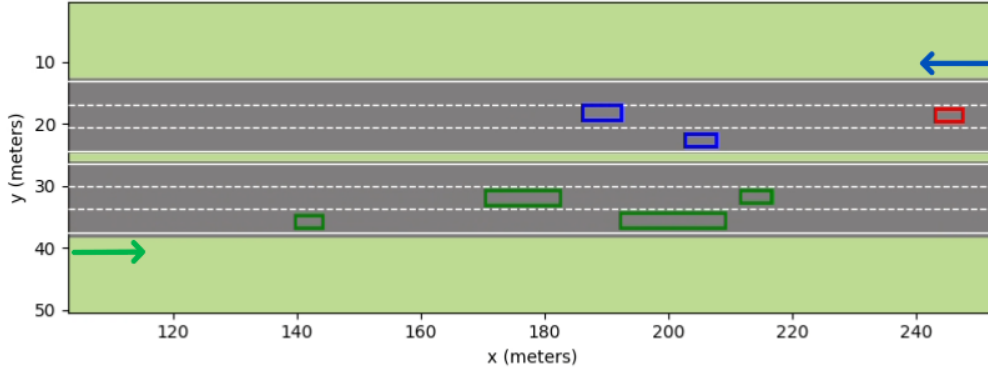


Figure 4.2.: Example traffic scene from a selected highD subset. The red box denotes the ego vehicle, while the blue and green boxes represent other vehicles traveling in the two opposite driving directions.

For each vehicle, highD provides frame-level kinematic variables such as longitudinal and lateral position, velocity, and acceleration. It also provides lane-related information, driving-direction labels, and identifiers for neighboring vehicles in 8 surrounding positions. In addition, the dataset provides traffic-safety indicators such as TTC and THW, which support the construction of interaction and risk-related features. These signals form the basis for the preprocessing and feature-engineering pipeline described in Section 4.3.

For the selected subset, 50,543 vehicle trajectories are available after loading the chosen recordings. With a window length of $W = 50$ frames, 50,455 trajectories satisfy $T \geq W$, while only 88 are shorter than the required window length. Most trajectories are long enough for temporal modeling: half have about 349 frames, and 80% fall between 280 and 466 frames. This indicates that the selected subset is sufficiently large and temporally rich for constructing window-based observation sequences.

4.3. Data Preparation and Preprocessing

4.3.1. Preprocessing Pipeline

The raw highD files are first merged into a single per-frame representation that combines trajectory data with vehicle-level and recording-level metadata. Since the dataset contains multiple recordings, each trajectory is identified by the pair $(\text{recording_id}, \text{vehicle_id})$. Vehicle-center coordinates are then computed from the original bounding-box positions to provide a consistent geometric reference.

The selected recordings contain traffic moving in two opposite roadway directions. In the raw data, forward longitudinal motion has opposite signs across these directions, and the same inconsistency affects related kinematic variables. Therefore, the raw signals from highD cannot be used directly as a single consistent representation. A set of plots

4. Methodology

illustrating this effect is provided in Appendix A. To resolve this issue, a vehicle-centric normalization step is applied using the driving-direction label. This transformation makes the sign conventions for longitudinal and lateral motion consistent across both traffic directions. As a result, the same maneuver is represented with the same sign convention regardless of the original roadway direction.

After these normalization steps, the processed dataframe provides a consistent per-frame representation that is used for feature engineering and subsequent windowization.

4.3.2. Data Analysis for Feature Design

Before defining the final observation vector, an exploratory analysis of the highD variables was carried out to understand the statistical behavior of the available signals and to identify informative features. The analysis focused mainly on kinematic variables, namely longitudinal and lateral acceleration (a_x, a_y), longitudinal and lateral velocity (v_x, v_y), and simple lane-change indicators.

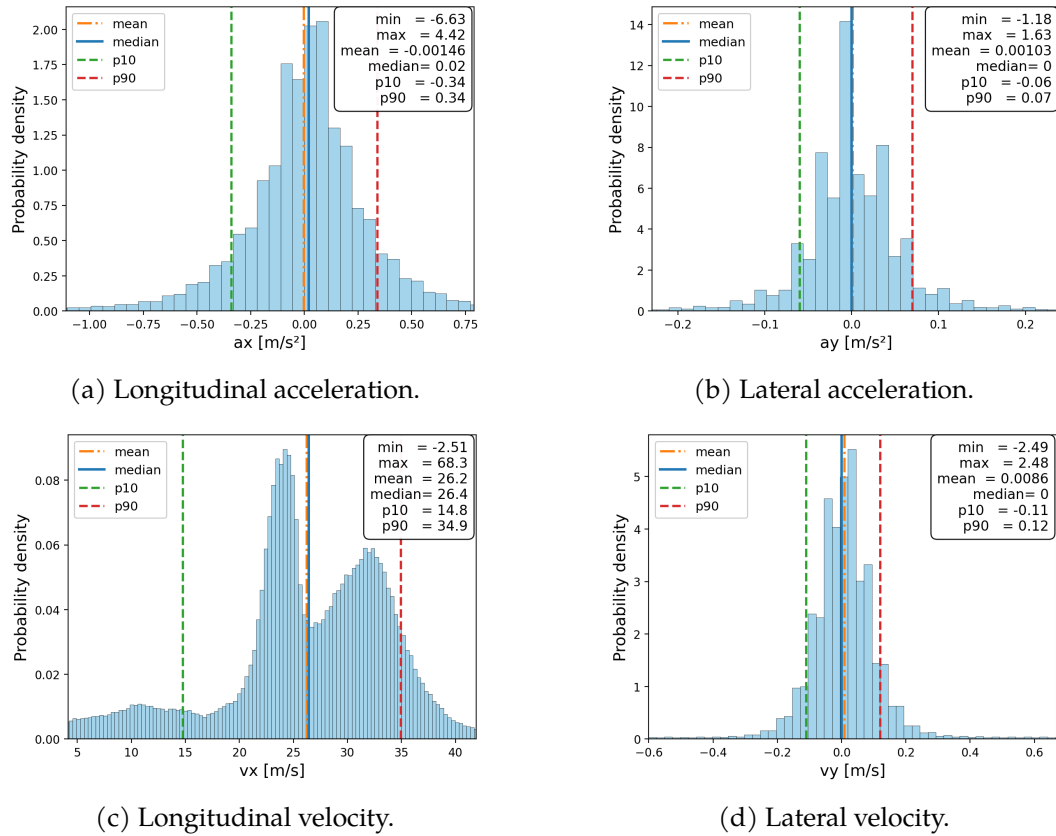


Figure 4.3.: Global distributions of the main kinematic variables.

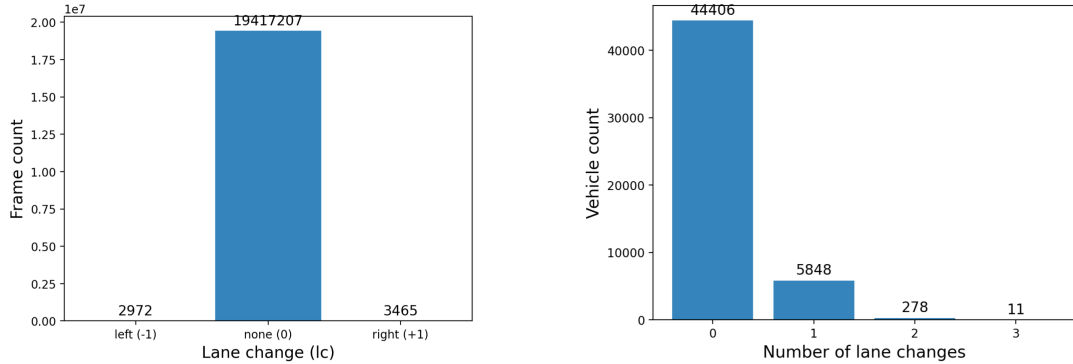
As shown in Figure 4.3a, the distribution of a_x is centered near zero but has visible tails on both sides, indicating that most frames correspond to near-steady motion, while

4. Methodology

stronger acceleration and braking occur less often. The longitudinal velocity distribution in Figure 4.3c spans a broad range of values and exhibits a multimodal structure. This is mainly caused by differences between vehicle classes, with trucks concentrated at lower speeds and cars occupying a wider and generally higher-speed range. These class-dependent differences show that raw continuous features can have different natural ranges depending on the vehicle type. Class-wise kinematic distributions for cars and trucks are provided in Appendix B. For this reason, continuous variables are normalized before training, as described in Section 4.5.

On the other hand, the lateral variables are much more concentrated around zero. In Figure 4.3b and Figure 4.3d, both a_y and v_y show sharp peaks near zero, which is expected because most vehicles remain within their lane for most of the time. However, the non-zero tails still reflect lateral adjustments and lane-change-related motion. Together, these observations support the inclusion of both longitudinal and lateral kinematic variables, since the former describe speed adaptation and car-following behavior, whereas the latter provide cues for lateral maneuvers.

In addition to the raw frame-level signals, the variability of each kinematic variable was also analyzed at trajectory level. For each vehicle trajectory, the standard deviation of a_x , a_y , v_x , and v_y was computed over time, producing one variability value per trajectory. Unlike the raw kinematic distributions, these plots do not describe the absolute values of the signals, but rather how strongly they fluctuate within individual trajectories. The resulting distributions are shown in Figure 4.5a to Figure 4.5d.



(a) Raw lane-change indicator at frame level.

(b) Number of lane changes observed per trajectory.

Figure 4.4.: Lane-change sparsity analysis in the highD data.

The variability analysis shows that longitudinal motion is generally more dynamic than lateral motion. In Figure 4.5a and Figure 4.5c, both $\text{std}(a_x)$ and $\text{std}(v_x)$ exhibit broader distributions with long right tails, indicating that some vehicles undergo substantial longitudinal speed and acceleration changes. In contrast, Figure 4.5b shows that $\text{std}(a_y)$ remains concentrated at low values, which is consistent with lateral acceleration being close to zero except during lane changes. A similar pattern appears in Figure 4.5d,

4. Methodology

although the visible tail suggests that some trajectories contain stronger lateral motion. Class-wise trajectory-level variability distributions are provided in Appendix B. These results support the use of variability-related window features, since instantaneous values alone do not capture whether a trajectory is stable or dynamically changing over time.

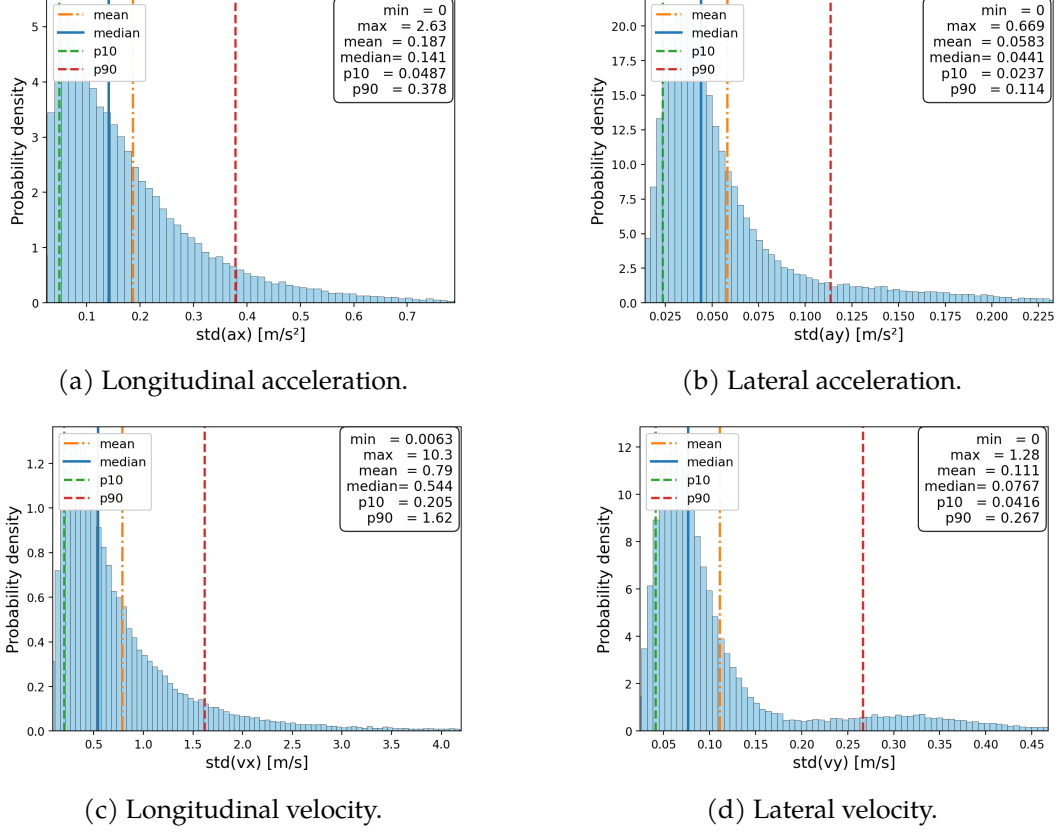


Figure 4.5.: Global trajectory-level variability distributions of the main kinematic variables.

The lane-change analysis further highlights an important property of the dataset. As shown in Figure 4.4a, almost all frames belong to the lane-keeping class, while left and right lane changes occur only rarely. The same sparsity appears at trajectory level in Figure 4.4b, where most vehicles perform no lane change and only a small fraction perform one or more. This imbalance shows that lane-change events are very rare compared to straight driving. It also shows that the raw lane-change indicator alone is not sufficient as a learning signal, especially because in highD the lane change is marked only at the frame where the vehicle crosses the lane boundary. This motivated the use of temporal aggregation together with contextual kinematic and interaction features.

Overall, the data analysis led to three main conclusions. First, longitudinal and lateral kinematics provide complementary information for maneuver prediction. Second,

trajectory-level variability contains useful information beyond single-frame values. Third, feature scale and class-dependent distribution differences must be handled carefully during preprocessing. These findings guided the final feature design for the proposed DBN.

4.3.3. Feature Engineering

In the feature-engineering stage, the available neighboring positions include the front, rear, left-front, left-side, left-rear, right-front, right-side, and right-rear slots. For each available neighbor, binary existence flags are created and relative kinematic quantities are derived in the form of relative longitudinal distance, relative lateral distance, relative longitudinal velocity, and relative lateral velocity. Missing neighbors remain marked through the corresponding existence indicators.

In the final observation vector, only the subset of neighboring vehicles shown in Figure 4.6 is retained. This choice makes the model focus on the vehicles that are most relevant for immediate vehicle-following constraints in the current lane. It also captures the local traffic state of the adjacent lanes while keeping the feature space compact and less complex. Rear and rear-diagonal neighbors may also provide useful information, but they are excluded here to keep the observation space lower-dimensional.

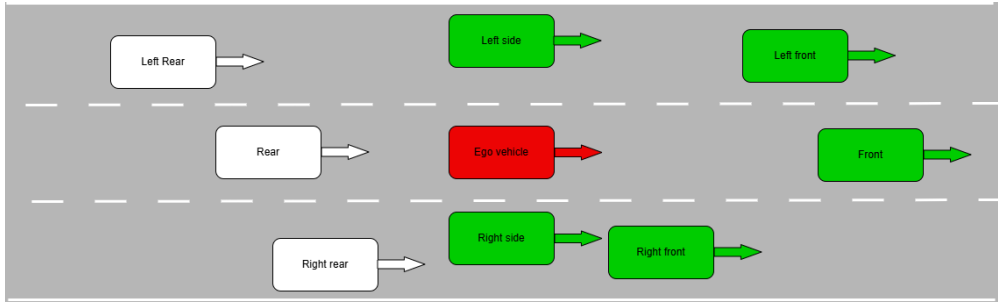


Figure 4.6.: Selected neighbors considered in the observation are marked in green.

In addition to interaction features, lane-related and risk-related quantities are derived. Distances to the left and right lane boundaries are computed from the recording metadata and the ego-vehicle center position. The lane-change indicator is obtained from changes in lane ID and represented as a ternary variable corresponding to lane change to left, no lane change, and lane change to right. The front-interaction safety signals provided by highD are mapped to the variables `front_thw` and `front_ttc`. Furthermore, ego-vehicle jerk is computed from the frame-to-frame change in longitudinal and lateral acceleration.

Based on the preprocessing and feature-engineering steps described above, the final frame-level observation was defined as the compact feature set listed in Table 4.1. The selected variables describe ego motion, lane-change evidence, lane geometry, local interaction context, and front-interaction risk. This frame-level representation serves as the basis for the subsequent window-based observation model.

Table 4.1.: Frame-level feature groups used in the preprocessing pipeline

Feature group	Variables	Purpose
Ego kinematics	$v_x, v_y, a_x, a_y, \text{jerk}_x, \text{jerk}_y$	Describe recent ego motion and control variation
Lane-change evidence	lc	Encode left / none / right lane-change evidence
Lane geometry	$d_{\text{left_lane}}, d_{\text{right_lane}}$	Represent lateral position within the current lane
Front risk	$\text{front_thw}, \text{front_ttc}$	Describe front-interaction criticality
Neighbor existence	Existence indicators for the selected front and adjacent-lane neighbor slots	Indicate whether relevant surrounding vehicles are present
Relative interaction	For each selected neighbor slot: dx, dy, dv_x, dv_y	Capture relative position and motion with respect to surrounding traffic

4.3.4. Window-Based Representation

The proposed DBN operates on short temporal windows rather than on individual frames. Each vehicle trajectory is therefore converted into a sequence of overlapping windows, as illustrated in Figure 4.7. This allows the model to use recent motion history and interaction context instead of relying only on instantaneous frame values.

Let W denote the window length and S the stride between consecutive windows. In this work, $W = 50$ frames and $S = 10$ frames are used. Since highD is recorded at 25 Hz, each window covers approximately 2 s of history, while consecutive windows start 0.4 s apart. Thus, for a trajectory of length T frames, overlapping windows are extracted every S frames as long as $T \geq W$. Each resulting observation summarizes the selected features over one interval of W consecutive frames.

Each window is converted into a single observation vector by aggregating the frame-level signals over the corresponding interval. Depending on the variable, the aggregation uses the most recent value, a minimum over the window, a temporal slope, sign-based fractions, percentile-based summaries, binary event indicators, or fractions of valid or available frames. For neighbor-dependent variables, the summaries are computed only when the corresponding neighbor is present. The complete list of window-level features used in the final observation vector is provided in Appendix C.

4.4. Proposed DBN Model

The proposed model represents driver behavior through two discrete latent variables: a style state S_t and an action state A_t . The style state captures a broader driving regime,

4. Methodology

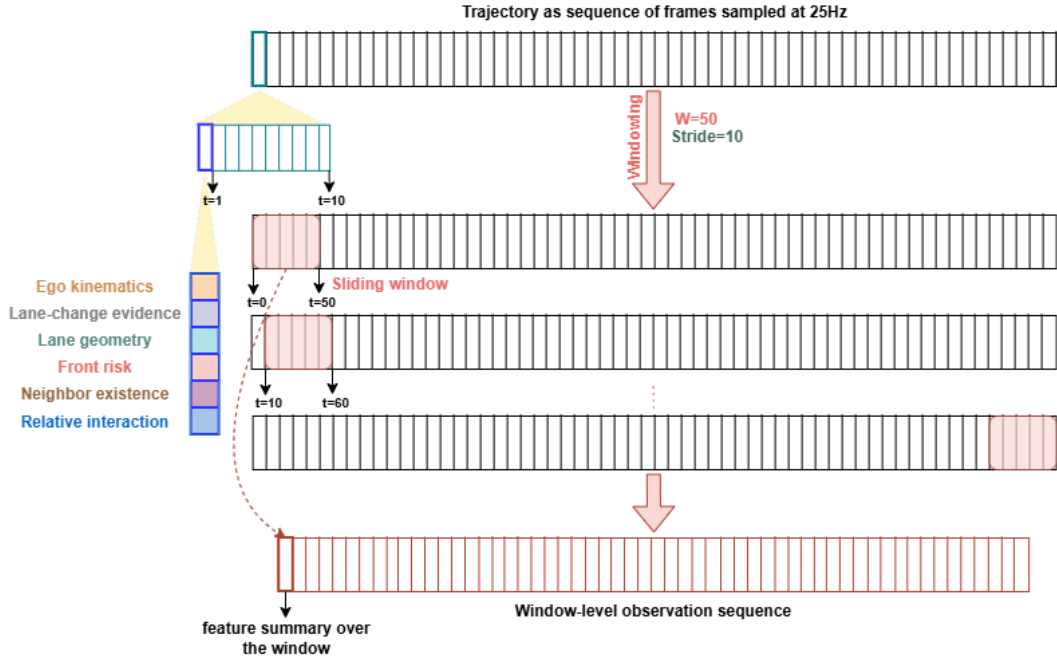


Figure 4.7.: Illustration of the sliding-window representation used in this work.

while the action state represents the maneuver at the current time step. The observation vector O_t corresponds to the window-level features. Notation follows the convention introduced in paragraph 2.

4.4.1. Latent State Structure

Rather than using a single latent variable, the model uses a factorized latent representation. As shown in Figure 4.8, the style state evolves as its own Markov chain, while the action state depends on both the previous action and the current style state.

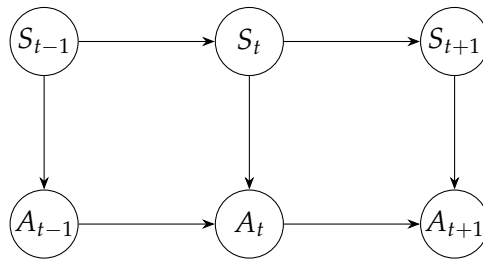


Figure 4.8.: Factorized latent structure of the proposed DBN.

4. Methodology

4.4.2. Transition Model

The initial distribution is modeled as:

$$p(S_0 = s_0) = \pi_{s_0}^{(s)} \quad (4.1)$$

$$p(A_0 = a_0 \mid S_0 = s_0) = \Pi_{s_0, a_0}^{(a)} \quad (4.2)$$

For later time steps, the transition model is factorized as:

$$p(S_t = s_t \mid S_{t-1} = s_{t-1}) = A_{s_{t-1}, s_t}^{(s)} \quad (4.3)$$

$$p(A_t = a_t \mid A_{t-1} = a_{t-1}, S_t = s_t) = \mathcal{A}_{s_t, a_{t-1}, a_t}^{(a)} \quad (4.4)$$

Here, $\pi^{(s)}$ is the initial style-probability vector, $\Pi^{(a)}$ is the conditional initial action matrix, $A^{(s)}$ is the style transition matrix, and $\mathcal{A}^{(a)}$ is the action transition tensor.

This factorization reflects the assumption that the style state evolves more gradually, while the current maneuver depends on both the recent maneuver history and the current driving regime.

4.4.3. Emission Model Variants

At each time step, the emission model specifies the conditional probability of the window-level observation vector given the latent state pair (S_t, A_t) . Two alternative emission formulations are considered in this work, as illustrated in Figure 4.9. Both share the same latent transition structure and differ only in how the observation model is parameterized. In both cases, the observation vector is split into continuous and binary components, modeled by diagonal Gaussian and Bernoulli distributions, respectively.

Product-of-Experts Emission

In the Product of Experts (PoE) formulation, the observation likelihood is decomposed into two experts: one conditioned on the style state and one conditioned on the action state. Their contributions are combined multiplicatively and then normalized:

$$p(O_t \mid S_t = s_t, A_t = a_t) = \frac{p_s(O_t \mid S_t = s_t; \theta_{s_t}) p_a(O_t \mid A_t = a_t; \theta_{a_t})}{Z_{s_t, a_t}} \quad (4.5)$$

Here, θ_{s_t} denotes the parameters of the style expert associated with style state s_t , and θ_{a_t} denotes the parameters of the action expert associated with action state a_t . Each parameter set contains the Gaussian parameters for the continuous features and the Bernoulli parameters for the binary features. The term Z_{s_t, a_t} is the normalization constant that ensures the resulting distribution is valid.

4. Methodology

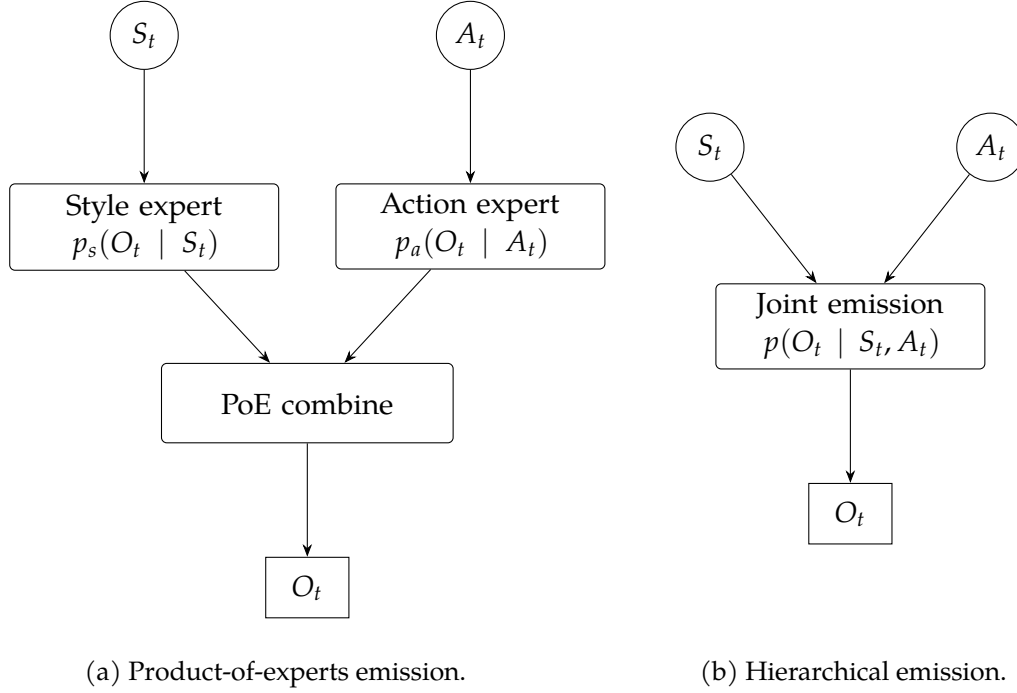


Figure 4.9.: Emission model variants considered in the proposed DBN.

Hierarchical Emission

In the hierarchical formulation, a single emission distribution is learned directly for each joint latent-state pair:

$$p(O_t | S_t = s_t, A_t = a_t) = p(O_t | S_t = s_t, A_t = a_t; \theta_{s_t, a_t}) \quad (4.6)$$

Here, θ_{s_t, a_t} denotes the parameters of the emission distribution associated with the joint state pair (s_t, a_t) . As in the PoE case, these parameters include the Gaussian terms for the continuous features and the Bernoulli terms for the binary features. This formulation gives the model greater flexibility for each joint latent-state pair, at the cost of a larger number of emission parameters.

4.4.4. Modeling Assumptions

The proposed DBN model is based on the following modeling assumptions:

- **First-order temporal dependence:** The style state S_t depends only on the previous style state S_{t-1} . The action state A_t depends only on the previous action state A_{t-1} and the current style state S_t . This gives a first-order Markov structure for the latent process.
- **Observation dependence on the current latent state only:** The observation vector O_t depends only on the latent variables at the same time step. Once S_t and A_t are

known, the current observation is assumed to be conditionally independent of past observations and past latent states.

- **Window-level observation representation:** Each model time step corresponds to one windowed observation vector rather than one raw frame from the dataset. Therefore, the model predicts behavior from short temporal summaries of recent motion, interaction, and risk-related features.
- **Split into continuous and binary features:** The observation vector is divided into continuous and binary components. Among the window features, the lane-change indicator variables are binary, while the remaining features are treated as continuous.
- **Diagonal Gaussian assumption for continuous features:** Continuous features are modeled using diagonal Gaussian distributions. This means that, conditioned on the relevant latent state, the continuous feature dimensions are treated as independent.
- **Bernoulli assumption for binary features:** Binary features are modeled using Bernoulli distributions. The binary dimensions are treated as conditionally independent given the relevant latent state.
- **Missing-feature handling:** Some features may be undefined at certain time steps, especially interaction-related quantities when neighboring vehicles are absent. Such values are treated as missing and are excluded from the corresponding likelihood terms through masking, rather than being interpreted as informative values.

4.5. Model Training

As described in Section 4.1, the windowized vehicle sequences are divided into training, validation, and test sets. This section explains how the observation sequences are prepared, how the model parameters are initialized, and how the proposed DBN is trained and monitored during learning.

4.5.1. Feature Scaling

Before training, the continuous window-level features are normalized to make them comparable in scale. This is necessary because the observation vector contains different physical quantities, such as velocity, acceleration, lane-relative distances, and interaction-related measures, which naturally have different numerical ranges. Without scaling, features with larger magnitudes could dominate the likelihood computation and make parameter estimation less stable.

In the implemented pipeline, z-score scaling statistics, namely the mean and standard deviation, are computed from the training set and then applied unchanged to the

validation and test sets. The same transformation is reused during inference when the learned model is applied to new observation sequences. Only continuous features are normalized in this way. The binary lane-change indicators, namely `lc_left_present` and `lc_right_present`, are kept in their original form. These scaling values are fixed preprocessing quantities and are not part of the learned emission model.

4.5.2. Parameter Initialization

After feature scaling, the model parameters are initialized before iterative learning begins. This includes both the temporal parameters of the DBN and the parameters of the emission model. A suitable initialization is important because the EM algorithm is sensitive to the starting point and may otherwise converge to a poor local optimum.

In the implemented framework, the emission parameters are initialized from the scaled training observations using K-means clustering. For the continuous part of the observation vector, the cluster centers and cluster variances are used as the initial Gaussian emission parameters. For the binary part, the initial Bernoulli probabilities are derived from the clustered observations. This provides a more informative starting point than a purely random initialization.

The initial latent-state distribution and the transition probabilities are also initialized before the first EM iteration. Depending on the experiment, different initialization schemes can be used, such as uniform, random, or sticky initialization. Here, sticky initialization means that remaining in the same latent state is given a higher initial probability than switching to a different state.

4.5.3. EM-Based Training

After initialization, the DBN is trained iteratively on the windowized observation sequences. At each iteration, the current model is applied to all training sequences to compute the posterior state probabilities and expected transition counts. These quantities are accumulated over all sequences and then used to update the model parameters. The procedure is repeated until the stopping criterion is met.

The re-estimation of the initial-state and transition parameters follows the EM update equations introduced in Section 2.4. The emission-parameter update depends on the chosen observation model. The theoretical derivations are given in Appendix D.

In the hierarchical emission model, one emission distribution is learned directly for each joint latent-state pair (S_t, A_t) . After the E-step, the posterior probabilities of the joint states are used as weights for re-estimating the emission parameters. For the continuous features, posterior-weighted means and diagonal variances are computed. For the binary features, posterior-weighted Bernoulli probabilities are computed. Thus, the hierarchical emission parameters can be updated in closed form.

In the PoE emission model, because of the normalization term, the style and action emission parameters are coupled during learning. As a result, the M-step does not reduce

Algorithm 1 EM-based training procedure**Require:** scaled training and validation sequences, initialized model parameters

```

1: repeat
2:   for each training sequence do
3:     compute emission log-likelihoods
4:     run forward-backward inference
5:     accumulate posterior state probabilities and expected transition counts
6:   end for
7:   update initial-state, transition, and emission parameters
8:   evaluate the current model on the validation set
9: until stopping criterion is satisfied

```

to a closed-form update as in the hierarchical case. Instead, the PoE emission parameters are updated by an inner gradient-based optimization loop within each EM iteration.

Lane-change-related events are much rarer than lane-keeping behavior, as shown in Figure 4.4. For this reason, additional weighting was investigated during training so that rare observations could have a stronger influence on learning. This follows the general idea of cost-sensitive weighting for imbalanced data [65]. In this case, lane-change-active time steps are upweighted using an inverse-frequency rule computed from the training data, with an upper clipping threshold to avoid excessively large weights.

Two weighting variants were considered. In the first variant, the weights are applied directly to the emission log-likelihood before forward-backward inference. Since multiplying a log-likelihood term by a positive weight is equivalent to raising the corresponding likelihood term to a power, this variant can be interpreted as a power-likelihood or tempered-likelihood formulation [66]. As a result, this variant modifies the posterior inference itself.

In the second variant, forward-backward inference is first carried out under the original model, and the weights are applied afterward to the posterior quantities used in the parameter update. This is closer to weighted-data EM, in which observations contribute to parameter estimation with different strengths through weighted responsibilities and weighted update equations [67]. When weighting is enabled, the inference step remains unchanged, and only the parameter update is weighted.

After each iteration, the model is evaluated on the validation set using the average validation log-likelihood per observation. Training stops when this value no longer improves meaningfully over several successive iterations.

4.6. Inference and Evaluation

After training, the model is applied to unseen sequences from the held-out test split. Evaluation is carried out in two complementary settings. First, full-sequence inference is

4. Methodology

used to assess how well the trained DBN explains unseen observation sequences. Second, causal online inference is used to predict the next latent state from observations available up to the current time step.

Across the experimental study, multiple model configurations are first compared using generative evaluation metrics on unseen observation sequences. Based on this comparison, a smaller set of candidate models is then selected for predictive evaluation in a strictly causal online setting.

For full-sequence analysis, inference is performed over the complete observation sequence to obtain the smoothed joint posterior distribution

$$p(S_t, A_t \mid O_{1:T}), \quad (4.7)$$

for each time step t . Since this posterior is conditioned on the full sequence, including future observations relative to time t , it is not a causal prediction setting. Instead, it is used to evaluate how well the model explains unseen data under the learned probabilistic structure. A better generative fit indicates that the model captures the temporal and contextual structure of driver behavior more faithfully, not just the next-step prediction.

In this work, generative fit is assessed using likelihood-based measures such as average log-likelihood per time step and Bayesian Information Criterion (BIC), together with posterior-based diagnostics such as entropy and state occupancy.

For predictive evaluation, inference is performed online. At time step t , the current observation O_t is used to update the filtered belief $p(S_t, A_t \mid O_{1:t})$, and the transition model is then used to compute the one-step-ahead predictive distribution. Using the transition factorization from Section 4.4.2, this is given by:

$$p(S_{t+1} = s', A_{t+1} = a' \mid O_{1:t}) = \sum_{s_t} \sum_{a_t} p(S_{t+1} = s' \mid S_t = s_t) p(A_{t+1} = a' \mid A_t = a_t, S_{t+1} = s') p(S_t = s_t, A_t = a_t \mid O_{1:t}) \quad (4.8)$$

The one-step prediction is then taken as the Maximum a Posteriori (MAP) estimate of the joint latent state:

$$(\hat{S}_{t+1}, \hat{A}_{t+1}) = \arg \max_{s, a} p(S_{t+1} = s, A_{t+1} = a \mid O_{1:t}). \quad (4.9)$$

During online evaluation, each test trajectory is processed sequentially. The first few windows are treated as a warmup period and are not scored, so that the predictor first accumulates sufficient recent context. After this warmup period, the model outputs a one-step-ahead prediction at each valid time step. These predictions are compared with rule-based reference labels derived from observable features. Figure 4.10 illustrates the causal prediction and evaluation procedure used in this work.

4. Methodology

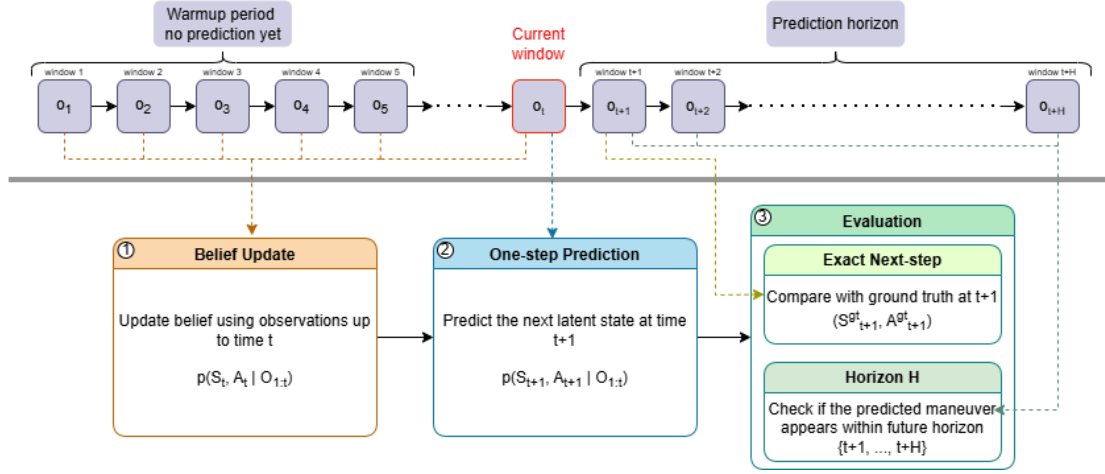


Figure 4.10.: Causal online prediction and evaluation procedure.

4.6.1. Generative Evaluation Metrics

Average Log-Likelihood per Time Step

For a set of test sequences indexed by i , the average log-likelihood per time step is defined as:

$$\bar{\ell}_{\text{step}} = \frac{1}{\sum_i T_i} \sum_i \log p(O_{1:T_i}^{(i)}) \quad (4.10)$$

where T_i is the length of sequence i .

This metric measures how well the trained model explains unseen observation sequences on average. Since total log-likelihood scales with sequence length, normalizing by $\sum_i T_i$ makes the score comparable across trajectories of different lengths. Higher values, that is, values that are less negative, indicate a better fit. For the proposed DBN, the likelihood of a sequence is obtained by marginalizing over all latent style and action sequences:

$$p(O_{1:T_i}^{(i)}) = \sum_{s_{1:T_i}} \sum_{a_{1:T_i}} p(O_{1:T_i}^{(i)}, s_{1:T_i} = s_{1:T_i}, a_{1:T_i} = a_{1:T_i}). \quad (4.11)$$

In practice, this quantity is evaluated with the forward algorithm rather than by explicit summation over all latent-state sequences [10], [13].

To complement the global average score, a normalized log-likelihood is also computed for each vehicle trajectory:

$$\ell_i = \frac{1}{T_i} \log p(O_{1:T_i}^{(i)}). \quad (4.12)$$

The distribution of ℓ_i over the test split is then summarized using percentiles such as the 5th, 50th, and 95th percentiles.

This diagnostic shows whether good average likelihood is obtained uniformly across vehicles or is dominated by only a subset of trajectories.

4. Methodology

Bayesian Information Criterion

To compare model variants with different numbers of free parameters, a complexity-penalized score based on the BIC is also computed on the training set:

$$\text{BIC} = k \log N - 2 \log \hat{L}, \quad (4.13)$$

where k is the number of free model parameters, $N = \sum_i T_i$ is the total number of training time steps, and $\hat{L}$ denotes the fitted likelihood on the training set used for model comparison. Lower BIC values indicate a better trade-off between goodness of fit and model complexity [68].

For the proposed model,

$$k = k_\pi + k_{\text{trans}} + k_{\text{em}}. \quad (4.14)$$

With K_S style states, K_A action states, D_c continuous features, and D_b binary features, the number of free parameters is

$$k_\pi = (K_S - 1) + K_S(K_A - 1), \quad (4.15)$$

$$k_{\text{trans}} = K_S(K_S - 1) + K_S K_A(K_A - 1). \quad (4.16)$$

For the hierarchical emission model,

$$k_{\text{em}}^{\text{hier}} = K_S K_A (2D_c + D_b), \quad (4.17)$$

assuming diagonal Gaussian parameters for the continuous part and Bernoulli parameters for the binary part. For the PoE emission model,

$$k_{\text{em}}^{\text{PoE}} = (K_S + K_A)(2D_c + D_b). \quad (4.18)$$

Posterior Entropy

The certainty of the inferred latent-state assignments is analyzed through normalized posterior entropy.

The normalized joint entropy is defined as:

$$H_{\text{joint}}^{(i)}(t) = -\frac{1}{\log(K_S K_A)} \sum_{s=1}^{K_S} \sum_{a=1}^{K_A} \gamma_t^{(i)}(s, a) \log \gamma_t^{(i)}(s, a), \quad (4.19)$$

with the convention that $0 \log 0 = 0$. This normalization ensures that

$$0 \leq H_{\text{joint}}^{(i)}(t) \leq 1. \quad (4.20)$$

The corresponding marginal style and action entropies are:

$$H_{\text{style}}^{(i)}(t) = -\frac{1}{\log K_S} \sum_{s=1}^{K_S} \gamma_t^{(i)}(s) \log \gamma_t^{(i)}(s), \quad (4.21)$$

$$H_{\text{action}}^{(i)}(t) = -\frac{1}{\log K_A} \sum_{a=1}^{K_A} \gamma_t^{(i)}(a) \log \gamma_t^{(i)}(a). \quad (4.22)$$

4. Methodology

Lower entropy indicates a more confident latent-state assignment, while higher entropy indicates greater ambiguity [69].

State Occupancy and Effective Number of States

Posterior state occupancies are analyzed to determine how strongly the learned latent states are used on the test split. The joint occupancy is defined as:

$$C_{\text{joint}}(s, a) = \sum_i \sum_{t=1}^{T_i} \gamma_t^{(i)}(s, a), \quad (4.23)$$

and the normalized occupancy distribution is

$$p_{\text{joint}}(s, a) = \frac{C_{\text{joint}}(s, a)}{\sum_{s', a'} C_{\text{joint}}(s', a')}. \quad (4.24)$$

Analogously, the marginal occupancies for style and action are:

$$C_{\text{style}}(s) = \sum_i \sum_{t=1}^{T_i} \gamma_t^{(i)}(s) \quad (4.25)$$

$$C_{\text{action}}(a) = \sum_i \sum_{t=1}^{T_i} \gamma_t^{(i)}(a) \quad (4.26)$$

with normalized versions:

$$p_{\text{style}}(s) = \frac{C_{\text{style}}(s)}{\sum_{s'} C_{\text{style}}(s')}, \quad (4.27)$$

$$p_{\text{action}}(a) = \frac{C_{\text{action}}(a)}{\sum_{a'} C_{\text{action}}(a')}. \quad (4.28)$$

A concentrated occupancy distribution indicates that posterior mass is dominated by a small subset of states, whereas a flatter distribution indicates more balanced use of the state space.

To further quantify how many latent states are effectively used, the effective number of states can be computed from the entropy of the occupancy distribution:

$$N_{\text{eff}}^{\text{joint}} = \exp \left(- \sum_{s=1}^{K_S} \sum_{a=1}^{K_A} p_{\text{joint}}(s, a) \log p_{\text{joint}}(s, a) \right). \quad (4.29)$$

Analogous definitions are used for style and action. Larger values indicate that posterior mass is spread across more latent states, while smaller values indicate that the model relies on only a few states [70].

4.6.2. Predictive Evaluation Metrics

For the selected candidate models, predictive evaluation is carried out in a strictly causal setting after an initial warmup period. One-step-ahead predictions are assessed using exact next-step accuracy, horizon-based hit rate, and time-to-event. Predictions whose rule-based reference labels are undefined are excluded from scoring.

Exact One-Step Prediction Accuracy

For each valid prediction time, the model produces a one-step-ahead prediction of the joint latent state $\hat{z}_{t+1} = (\hat{S}_{t+1}, \hat{A}_{t+1})$ after observing the sequence up to time t . The exact one-step prediction accuracy is defined as the fraction of scored time steps for which the predicted joint state matches the corresponding rule-based reference label exactly:

$$\text{Accuracy} = \frac{1}{N} \sum_{n=1}^N \mathbf{1}[\hat{z}_{t+1}^{(n)} = z_{t+1}^{(n)}], \quad (4.30)$$

where N is the total scored prediction instances pooled over all trajectories.

Since the joint latent space is multi-class and class frequencies can be imbalanced, macro-averaged precision, recall, and F1-score are also computed from the joint-state confusion matrix. These metrics complement accuracy by giving equal importance to all classes rather than being dominated by the most frequent states [71].

Hit Rate Within a Prediction Horizon

Exact next-step accuracy is intentionally strict, since it counts a prediction as correct only when the predicted latent state appears immediately at time $t + 1$. In behavior prediction, however, a forecast may still be useful when the predicted maneuver occurs a short time later. For this reason, a horizon-based hit metric is also used.

Let $\hat{z}_{t+1}$ denote the one-step-ahead predicted joint state. For a future horizon of H steps, the prediction is counted as a hit if the predicted state appears at least once in the interval from $t + 1$ to $t + H$:

$$\text{Hit@H}(t) = \mathbf{1}[\exists h \in \{1, \dots, H\} : z_{t+h} = \hat{z}_{t+1}]. \quad (4.31)$$

The hit rate is then obtained by averaging this indicator over all scored prediction times.

In addition to exact matching, the implemented evaluation can also credit semantically equivalent latent-state pairs when they correspond to the same higher-level maneuver group. This makes the horizon-based metric less brittle and better aligned with the intended behavior-prediction task.

Time-to-Event

To measure how early a correct prediction occurs within the prediction horizon, the Time To Event (TTE) is also computed. For a prediction made at time t , let $h^*(t)$ denote the first future step at which the predicted state appears:

$$h^*(t) = \min \{h \in \{1, \dots, H\} : z_{t+h} = \hat{z}_{t+1}\}. \quad (4.32)$$

If no hit occurs within the horizon, the prediction is treated as a miss.

For successful hits, the time-to-event in seconds is given by

$$\text{TTE}_{\text{sec}}(t) = h^*(t) \Delta t, \quad (4.33)$$

where Δt denotes the duration of one window step. With window stride measured in frames and dataset frame rate measured in frames per second, this is

$$\Delta t = \frac{\text{stride}}{\text{frame rate}}. \quad (4.34)$$

Mean, median, and percentile TTE values are reported over successful hits only.

4.6.3. Posterior-Based Interpretation of Individual Predictions

To interpret selected predictions locally, the model output can be decomposed into four parts: the prior belief, the emission likelihood, the posterior belief, and the one-step forecast. At a chosen time step t , the prior belief describes the latent-state distribution before the current observation is incorporated. The emission likelihood measures how compatible the current observation is with each latent-state pair. The posterior belief then represents the updated latent-state distribution after observing the current traffic context, and the one-step forecast propagates this posterior one step forward.

This decomposition makes it possible to analyse whether a prediction is driven mainly by the transition dynamics, by strong evidence from the current observation, or by a combination of both. For the winning posterior state, additional feature-group contribution scores are also computed in order to show which groups of observation features contribute most strongly to the local explanation.

4.6.4. Computational Efficiency

Since the intended application is online behavior prediction, computational efficiency is also evaluated. For this purpose, the trained model is benchmarked under several execution settings, including single-vehicle prediction, multi-vehicle sequential prediction, and batched prediction. These benchmarks are carried out both in a model-only setting, where precomputed window observations are given directly to the predictor, and in an end-to-end setting, where the full pipeline from frame-level input through windowization, scaling, belief update, and prediction is included.

4. Methodology

For each benchmark setting, the latency of obtaining the first valid prediction is measured separately from steady-state subsequent prediction times. In the end-to-end setting, the warmup phase required to accumulate enough frames for the first valid window is also included. The reported statistics include the first-prediction latency as well as summary measures of subsequent prediction times, such as mean, median, and upper percentiles. These results are used to assess whether the proposed model is suitable for short-horizon online prediction in an AV context.

4.7. Semantic Interpretation of Latent States

After training, an additional analysis is performed to interpret the learned latent states. The goal of this step is to relate the latent style and action states to characteristic traffic patterns observed in the data. For this purpose, the trained model is applied again to the test split. For each windowized observation sequence, posterior inference is carried out to obtain the joint posterior probabilities $\gamma_t(s, a)$.

The semantic interpretation is based on posterior-weighted feature statistics computed from the original observation features. For a selected feature $o_t^{(f)}$ and a joint latent state pair (s, a) , the posterior-weighted mean is computed as:

$$\mu_{s,a,f} = \frac{\sum_n \sum_t \gamma_t^{(n)}(s, a) o_t^{(n,f)}}{\sum_n \sum_t \gamma_t^{(n)}(s, a)}. \quad (4.35)$$

where n indexes the analyzed vehicle trajectories. Posterior-weighted standard deviations are computed in the same way to describe the variability of each feature within the corresponding latent state pair.

$$\sigma_{s,a,f} = \left(\frac{\sum_n \sum_t \gamma_t^{(n)}(s, a) \left(o_t^{(n,f)} - \mu_{s,a,f} \right)^2}{\sum_n \sum_t \gamma_t^{(n)}(s, a)} \right)^{1/2}. \quad (4.36)$$

This produces a semantic feature profile for every joint state (s, a) , showing which motion, interaction, or risk-related properties are typically associated with that state.

5. Conclusions

Bibliography

- [1] M. R. Abedi, Z. Rashidi, N. Mokari, H. Saeedi, and N. Zorba, *Precise hdv positioning through safety-aware integrated sensing and communication in a value-of-information-driven 6g v2x system*, 2025. arXiv: 2510.02363 [eess.SY].
- [2] W. Schwarting, J. Alonso-Mora, and D. Rus, "Planning and decision-making for autonomous vehicles", *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, no. 1, pp. 187–210, 2018.
- [3] S. Lefevre, D. Vasquez, and C. Laugier, "A survey on motion prediction and risk assessment for intelligent vehicles", *Robomech Journal*, vol. 1, Jul. 2014.
- [4] N. A. Madjid, A. Ahmad, M. Mebrahtu, Y. Babaa, A. Nasser, S. Malik, B. Hassan, N. Werghi, J. Dias, and M. Khonji, "Trajectory prediction for autonomous driving: Progress, limitations, and future directions", *Information Fusion*, vol. 126, p. 103 588, Feb. 2026.
- [5] N. M. Negash and J. Yang, "Driver behavior modeling toward autonomous vehicles: Comprehensive review", *IEEE Access*, vol. 11, pp. 22 788–22 821, 2023.
- [6] A. Rudenko, L. Palmieri, M. Herman, K. M. Kitani, D. M. Gavrila, and K. O. Arras, "Human motion trajectory prediction: A survey", *The International Journal of Robotics Research*, vol. 39, no. 8, pp. 895–935, Jun. 2020.
- [7] Y. Yang, N. Negash, and J. Yang, "Recent advances in interactive driving of autonomous vehicles: Comprehensive review of approaches", *Automotive Innovation*, vol. 8, Apr. 2025.
- [8] K. Murphy, *Dynamic Bayesian Networks: Representation, Inference and Learning*. University of California, Berkeley, 2002.
- [9] R. Krajewski, J. Bock, L. Kloecker, and L. Eckstein, *The highd dataset: A drone dataset of naturalistic vehicle trajectories on german highways for validation of highly automated driving systems*, 2018. arXiv: 1810.05642 [cs.CV].
- [10] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [11] D. Koller and N. Friedman, *Probabilistic Graphical Models: Principles and Techniques* (Adaptive computation and machine learning). MIT Press, 2009.
- [12] K. P. Murphy, "Machine learning - a probabilistic perspective", in *Adaptive computation and machine learning series*, 2012.
- [13] L. Rabiner, "A tutorial on hidden markov models and selected applications in speech recognition", *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

Bibliography

- [14] S. Zhou, S. Zheng, J. Tian, R. Jiang, and H. M. Zhang, *Twenty-five years of the intelligent driver model: Foundations, extensions, applications, and future directions*, 2025. arXiv: 2506.05909 [physics.soc-ph].
- [15] M. Treiber, A. Hennecke, and D. Helbing, “Congested traffic states in empirical observations and microscopic simulations”, *Phys. Rev. E*, vol. 62, pp. 1805–1824, 2 Aug. 2000.
- [16] P. Gipps, “A behavioural car-following model for computer simulation”, *Transportation Research Part B: Methodological*, vol. 15, no. 2, pp. 105–111, 1981.
- [17] M. Bando, K. Hasebe, A. Nakayama, A. Shibata, and Y. Sugiyama, “Dynamical model of traffic congestion and numerical simulation”, *Phys. Rev. E*, vol. 51, pp. 1035–1042, 2 Feb. 1995.
- [18] R. Jiang, Q. Wu, and Z. Zhu, “Full velocity difference model for a car-following theory”, *Phys. Rev. E*, vol. 64, p. 017 101, 1 Jun. 2001.
- [19] A. Kesting, M. Treiber, and D. Helbing, “General lane-changing model mobil for car-following models”, *Transportation Research Record*, vol. 1999, pp. 86–94, 2007.
- [20] S. Wang, K. Gao, L. Zhang, Y. Liu, and L. Chen, *Probabilistic prediction of longitudinal trajectory considering driving heterogeneity with interpretability*, 2023. arXiv: 2312.12123 [cs.LG].
- [21] Q. Deng, J. Wang, and D. Soffker, “Prediction of human driver behaviors based on an improved hmm approach”, in *2018 IEEE Intelligent Vehicles Symposium (IV)*, 2018, pp. 2066–2071.
- [22] D. Chen, M. Zhu, H. Yang, X. Wang, and Y. Wang, *Data-driven traffic simulation: A comprehensive review*, 2023. arXiv: 2310.15975 [cs.LG].
- [23] J. Pearl and S. Russell, “Bayesian networks”, in *The Handbook of Brain Theory and Neural Networks*, M. A. Arbib, Ed., Cambridge, MA: MIT Press, 1995, pp. 149–153.
- [24] J. Li, B. Dai, X. Li, X. Xu, and D. Liu, “A dynamic bayesian network for vehicle maneuver prediction in highway driving scenarios: Framework and verification”, *Electronics*, vol. 8, no. 1, 2019.
- [25] D. Młynarczyk, G. Calvo, F. Palmi-Perales, C. Armero, V. Gómez-Rubio, and U. Martinez-Iranzo, *Bayesian network approach to building an affective module for a driver behavioural model*, 2025. arXiv: 2502.03254 [stat.AP].
- [26] K. K. Talluri, A. L. Madsen, and G. Weidl, *Enhancing safety standards in automated systems using dynamic bayesian networks*, 2025. arXiv: 2505.02050 [cs.AI].
- [27] S. Mozaffari, M. A. Sormoli, K. Koufos, and M. Dianati, *Multimodal manoeuvre and trajectory prediction for automated driving on highways using transformer networks*, 2023. arXiv: 2303.16109 [cs.LG].
- [28] C. Zhang, K. Chen, M. Zhu, H. Yang, and L. Sun, *Learning car-following behaviors using bayesian matrix normal mixture regression*, 2024. arXiv: 2404.16023 [stat.AP].

Bibliography

- [29] J. Bethge, M. Pfefferkorn, A. Rose, J. Peters, and R. Findeisen, “Model predictive control with gaussian-process-supported dynamical constraints for autonomous vehicles”, *ArXiv*, vol. abs/2303.04725, 2023.
- [30] R. Ghandour, A. J. Potams, I. Boulkaibet, B. Neji, and Z. Al Barakeh, “Driver behavior classification system analysis using machine learning methods”, *Applied Sciences*, vol. 11, no. 22, 2021.
- [31] Y. Bengio, A. Courville, and P. Vincent, *Representation learning: A review and new perspectives*, 2014. arXiv: 1206.5538 [cs.LG].
- [32] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
- [33] L. Xin, P. Wang, C.-Y. Chan, J. Chen, S. E. Li, and B. Cheng, *Intention-aware long horizon trajectory prediction of surrounding vehicles using dual lstm networks*, 2019. arXiv: 1906.02815 [cs.LG].
- [34] J. Hao, H. Xie, G. Fan, Y. Chen, and K. Song, “Vehicle trajectory prediction with driving style identification and intention fusion”, Oct. 2024, pp. 1–6.
- [35] S. Hochreiter and J. Schmidhuber, “Long short-term memory”, *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [36] F. Hui, C. Wei, W. ShangGuan, R. Ando, and S. Fang, “Deep encoder–decoder-nn: A deep learning-based autonomous vehicle trajectory prediction and correction model”, *Physica A: Statistical Mechanics and its Applications*, vol. 593, p. 126 869, 2022.
- [37] F. Diehl, T. Brunner, M. T. Le, and A. Knoll, *Graph neural networks for modelling traffic participant interaction*, 2019. arXiv: 1903.01254 [cs.LG].
- [38] A. Abdelraouf, R. Gupta, and K. Han, *Interaction-aware personalized vehicle trajectory prediction using temporal graph neural networks*, 2023. arXiv: 2308.07439 [cs.LG].
- [39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, *Attention is all you need*, 2023. arXiv: 1706.03762 [cs.CL].
- [40] V. Winland. “What is self-attention?”, IBM, Accessed: Oct. 30, 2025. [Online]. Available: <https://www.ibm.com/think/topics/self-attention>
- [41] J. Ngiam et al., *Scene transformer: A unified architecture for predicting multiple agent trajectories*, 2022. arXiv: 2106.08417 [cs.CV].
- [42] S. Shi, L. Jiang, D. Dai, and B. Schiele, *Motion transformer with global intention localization and local movement refinement*, 2023. arXiv: 2209.13508 [cs.CV].
- [43] Z. Zikang, L. Ye, J. Wang, K. Wu, and K. Lu, “Hivt: Hierarchical vector transformer for multi-agent motion prediction”, Jun. 2022, pp. 8813–8823.

Bibliography

- [44] K. Sohn, H. Lee, and X. Yan, “Learning structured output representation using deep conditional generative models”, in *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28, Curran Associates, Inc., 2015.
- [45] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, *Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data*, 2021. arXiv: 2001.03093 [cs.R0].
- [46] K. Mangalam, H. Girase, S. Agarwal, K.-H. Lee, E. Adeli, J. Malik, and A. Gaidon, *It is not the journey but the destination: Endpoint conditioned trajectory prediction*, 2020. arXiv: 2004.02025 [cs.CV].
- [47] H. Zhou, D. Ren, X. Yang, M. Fan, and H. Huang, “Csr: Cascade conditional variational auto encoder with socially-aware regression for pedestrian trajectory prediction”, *Pattern Recognition*, vol. 133, p. 109 030, 2023.
- [48] J. Ho, A. Jain, and P. Abbeel, *Denoising diffusion probabilistic models*, 2020. arXiv: 2006.11239 [cs.LG].
- [49] M. Peng, K. Chen, X. Guo, Q. Zhang, H. Zhong, M. Zhu, and H. Yang, *Diffusion models for intelligent transportation systems: A survey*, 2025. arXiv: 2409.15816 [eess.SY].
- [50] S. Lian, B. Zhou, S. Hu, J. Hu, G. Wang, J. Escribano, X. Na, and S. Jin, “Enhanced multimodal trajectory prediction for autonomous vehicles using advanced diffusion model techniques”, *2024 IEEE Intelligent Vehicles Symposium (IV)*, pp. 484–489, 2024.
- [51] Q. T. Liu, D. Li, S. S. Sohn, S. Yoon, M. Kapadia, and V. Pavlovic, “Trajdiffuse: A conditional diffusion model for environment-aware trajectory prediction”, in *Pattern Recognition*, A. Antonacopoulos, S. Chaudhuri, R. Chellappa, C.-L. Liu, S. Bhattacharya, and U. Pal, Eds., Cham: Springer Nature Switzerland, 2025, pp. 382–397.
- [52] C. M. Jiang, A. Cornman, C. Park, B. Sapp, Y. Zhou, and D. Anguelov, *Motiondiffuser: Controllable multi-agent motion prediction using diffusion*, 2023. arXiv: 2306.03083 [cs.R0].
- [53] S. Arora and P. Doshi, “A survey of inverse reinforcement learning: Challenges, methods and progress”, *Artificial Intelligence*, vol. 297, p. 103 500, 2021.
- [54] B. D. Ziebart, A. Maas, J. A. Bagnell, and A. K. Dey, “Maximum entropy inverse reinforcement learning”, in *Proceedings of the 23rd National Conference on Artificial Intelligence - Volume 3*, ser. AAAI’08, Chicago, Illinois: AAAI Press, 2008, pp. 1433–1438.
- [55] A. Y. Ng and S. J. Russell, “Algorithms for inverse reinforcement learning”, in *Proceedings of the Seventeenth International Conference on Machine Learning*, ser. ICML ’00, San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2000, pp. 663–670.

Bibliography

- [56] J. A. Rodrigues da Silva, V. Grassi, and D. F. Wolf, “Maximum entropy inverse reinforcement learning using monte carlo tree search for autonomous driving”, *Trans. Intell. Transport. Sys.*, vol. 25, no. 9, pp. 11 552–11 562, Sep. 2024.
- [57] S. Karimi and A. Vahidi, “Receding horizon motion planning for automated lane change and merge using monte carlo tree search and level-k game theory”, in *2020 American Control Conference (ACC)*, 2020, pp. 1223–1228.
- [58] H. Niu, Q. Chen, Y. Li, Y. Zhang, and J. Hu, *Stackelberg driver model for continual policy improvement in scenario-based closed-loop autonomous driving*, 2023. arXiv: 2309.14235 [cs.LG].
- [59] K. Ji, M. Orsag, and K. Han, “Lane-merging strategy for a self-driving car in dense traffic using the stackelberg game approach”, *Electronics*, vol. 10, no. 8, 2021.
- [60] Z. Huang, T. Li, S. Shen, and J. Ma, *Integrated decision making and trajectory planning for autonomous driving under multimodal uncertainties: A bayesian game approach*, 2024. arXiv: 2409.13993 [cs.R0].
- [61] C. Hubmann, M. Becker, D. Althoff, D. Lenz, and C. Stiller, “Decision making for autonomous driving considering interaction and uncertain prediction of surrounding vehicles”, in *2017 IEEE Intelligent Vehicles Symposium (IV)*, 2017, pp. 1671–1678.
- [62] L. Westhofen, C. Neurohr, T. Koopmann, M. Butz, B. Schütt, F. Utesch, B. Neurohr, C. Gutenkunst, and E. Böde, “Criticality metrics for automated driving: A review and suitability analysis of the state of the art”, *Archives of Computational Methods in Engineering*, vol. 30, no. 1, pp. 1–35, Aug. 2022.
- [63] X. Xiong, S. Zhang, and Y. Chen, “Review of intelligent vehicle driving risk assessment in multi-vehicle interaction scenarios”, *World Electric Vehicle Journal*, vol. 14, no. 12, 2023.
- [64] T. Nyberg, C. Pek, L. Col, C. Norén, and J. Tumova, “Risk-aware motion planning for autonomous vehicles with safety specifications”, May 2021.
- [65] J. He and M. Cheng, “Weighting methods for rare event identification from imbalanced datasets”, *Frontiers in Big Data*, vol. 4, Dec. 2021.
- [66] C. Holmes and S. Walker, *Assigning a value to a power likelihood in a general bayesian model*, 2017. arXiv: 1701.08515 [stat.ME].
- [67] I. D. Gebru, X. Alameda-Pineda, F. Forbes, and R. Horaud, “Em algorithms for weighted-data clustering with application to audio-visual scene analysis”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 12, pp. 2402–2415, Dec. 2016.
- [68] G. Schwarz, “Estimating the dimension of a model”, *Annals of Statistics*, vol. 6, pp. 461–464, 1978.

Bibliography

- [69] “Entropy, relative entropy, and mutual information”, in *Elements of Information Theory*. John Wiley & Sons, Ltd, 2005, ch. 2, pp. 13–55. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/047174882X.ch2>.
- [70] L. Jost, “Entropy and diversity”, *Oikos*, vol. 113, no. 2, pp. 363–375, 2006. eprint: <https://nsojournals.onlinelibrary.wiley.com/doi/pdf/10.1111/j.2006.0030-1299.14714.x>.
- [71] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks”, *Inf. Process. Manag.*, vol. 45, pp. 427–437, 2009.
- [72] A. P. Dempster, N. M. Laird, and D. B. Rubin, “Maximum likelihood from incomplete data via the em algorithm”, *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 39, no. 1, pp. 1–22, Dec. 2018. eprint: https://academic.oup.com/jrsssb/article-pdf/39/1/1/49117094/jrsssb_39_1_1.pdf.
- [73] J. Bilmes, “A gentle tutorial of the EM algorithm and its application to parameter estimation for Gaussian mixture and hidden Markov models”, *International Computer Science Institute*, vol. 4, p. 126, 1998.
- [74] G. E. Hinton, “Training products of experts by minimizing contrastive divergence”, *Neural Computation*, vol. 14, no. 8, pp. 1771–1800, 2002.
- [75] A. Wu, F. BYC., and W. Jeff, “On the convergence properties of the em algorithm”, *Annals of Statistics*, vol. 11, pp. 95–103, 1983.

A. Direction-Wise Sign Convention in Raw Kinematics

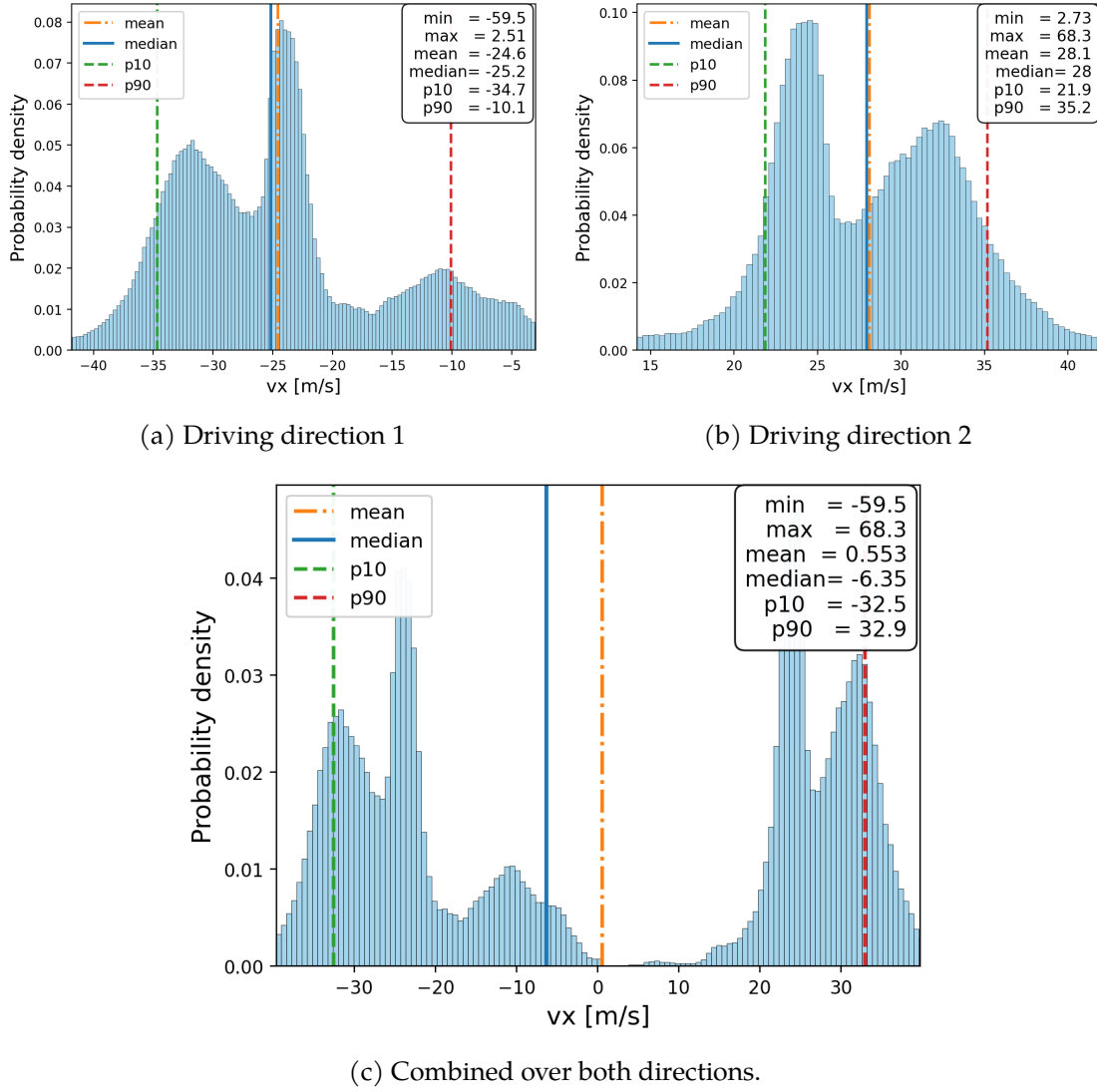


Figure A.1.: Direction-wise sign difference in raw longitudinal velocity.

A. Direction-Wise Sign Convention in Raw Kinematics

This appendix illustrates the sign inconsistency in raw highD kinematic signals caused by opposite traffic directions. For the longitudinal velocity v_x , forward motion appears with opposite signs in the two directions. As a result, the combined raw distribution is bimodal around zero and cannot be interpreted with a single sign convention.

For completeness, the same raw longitudinal velocity signal is also shown by vehicle class:

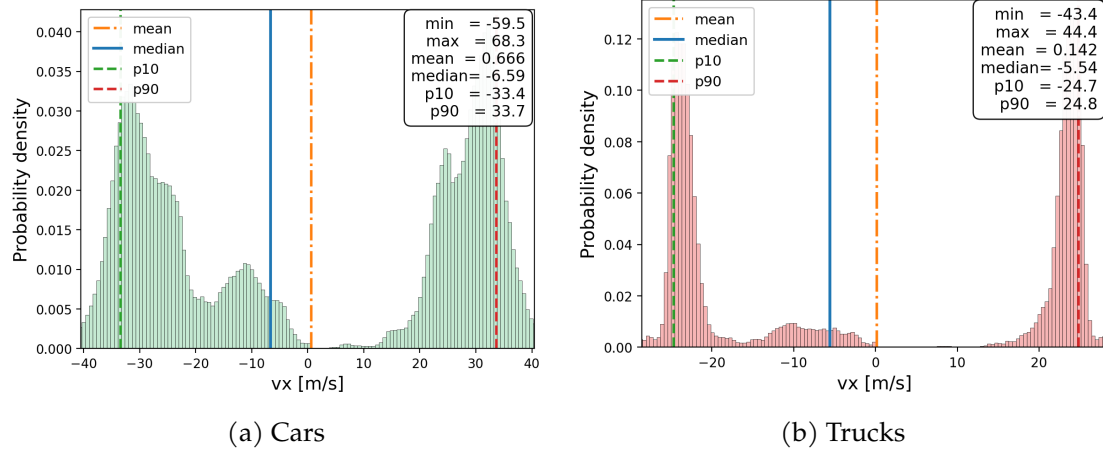


Figure A.2.: Class-wise raw longitudinal velocity distributions.

B. Class-Wise Distribution Plots

This appendix provides class-wise versions of the main kinematic and trajectory-level variability distributions used in the exploratory data analysis. The plots separate cars and trucks in order to make the class-dependent differences in scale and spread more explicit.

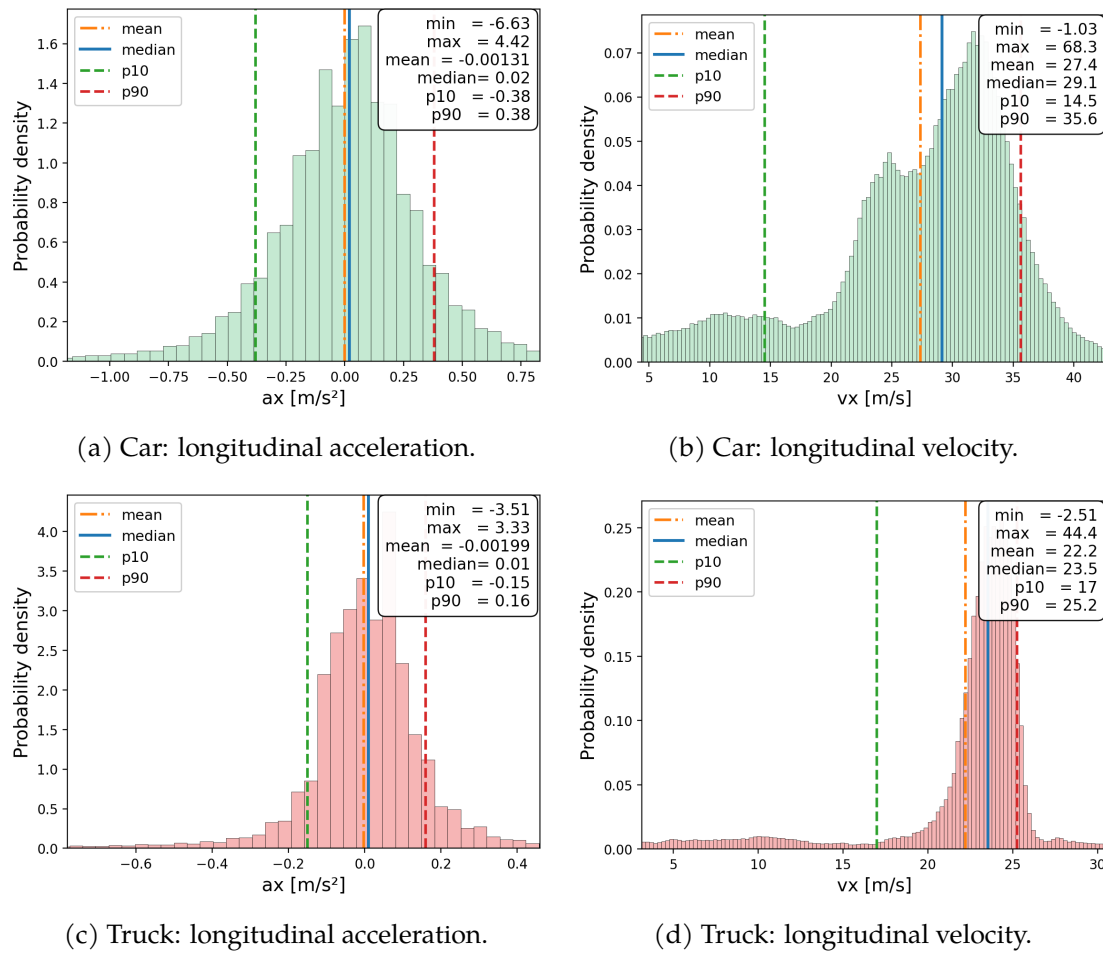
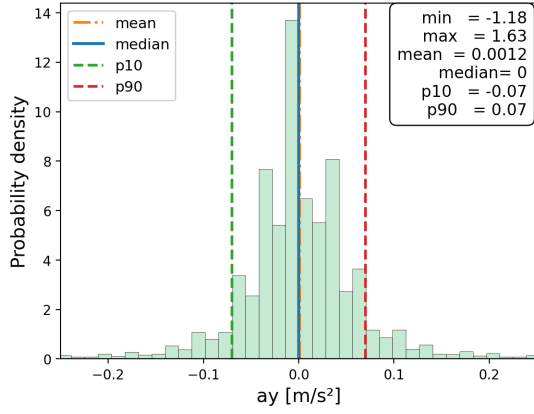
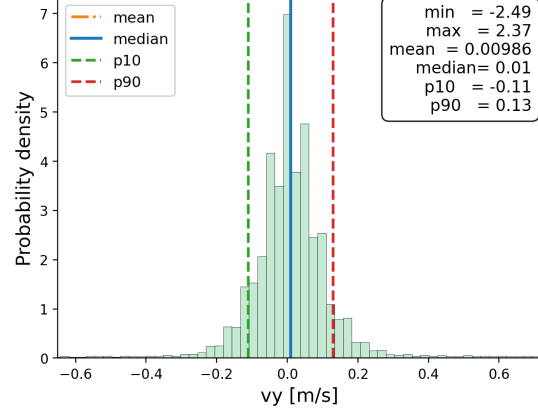


Figure B.1.: Class-wise frame-level distributions of the main kinematic variables.

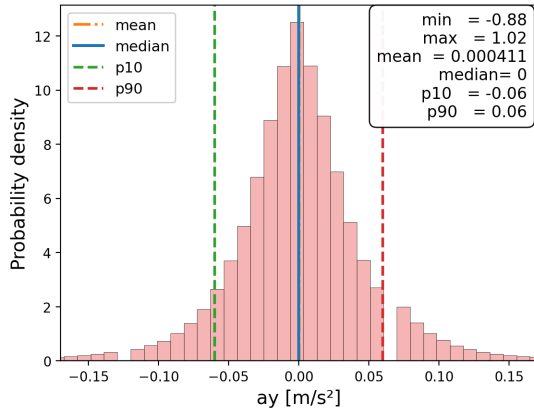
B. Class-Wise Distribution Plots



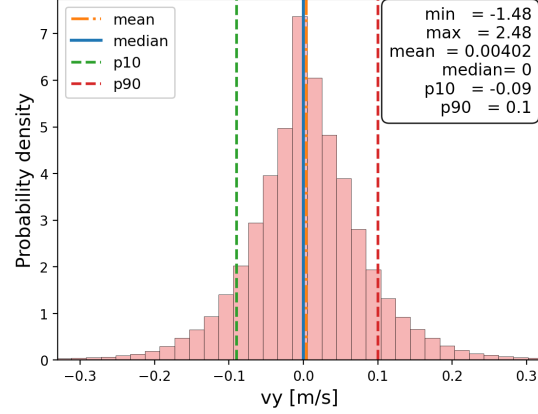
(e) Car: lateral acceleration.



(f) Car: lateral velocity.



(g) Truck: lateral acceleration.



(h) Truck: lateral velocity.

Figure B.1.: Class-wise frame-level distributions of the main kinematic variables (continued).

B. Class-Wise Distribution Plots

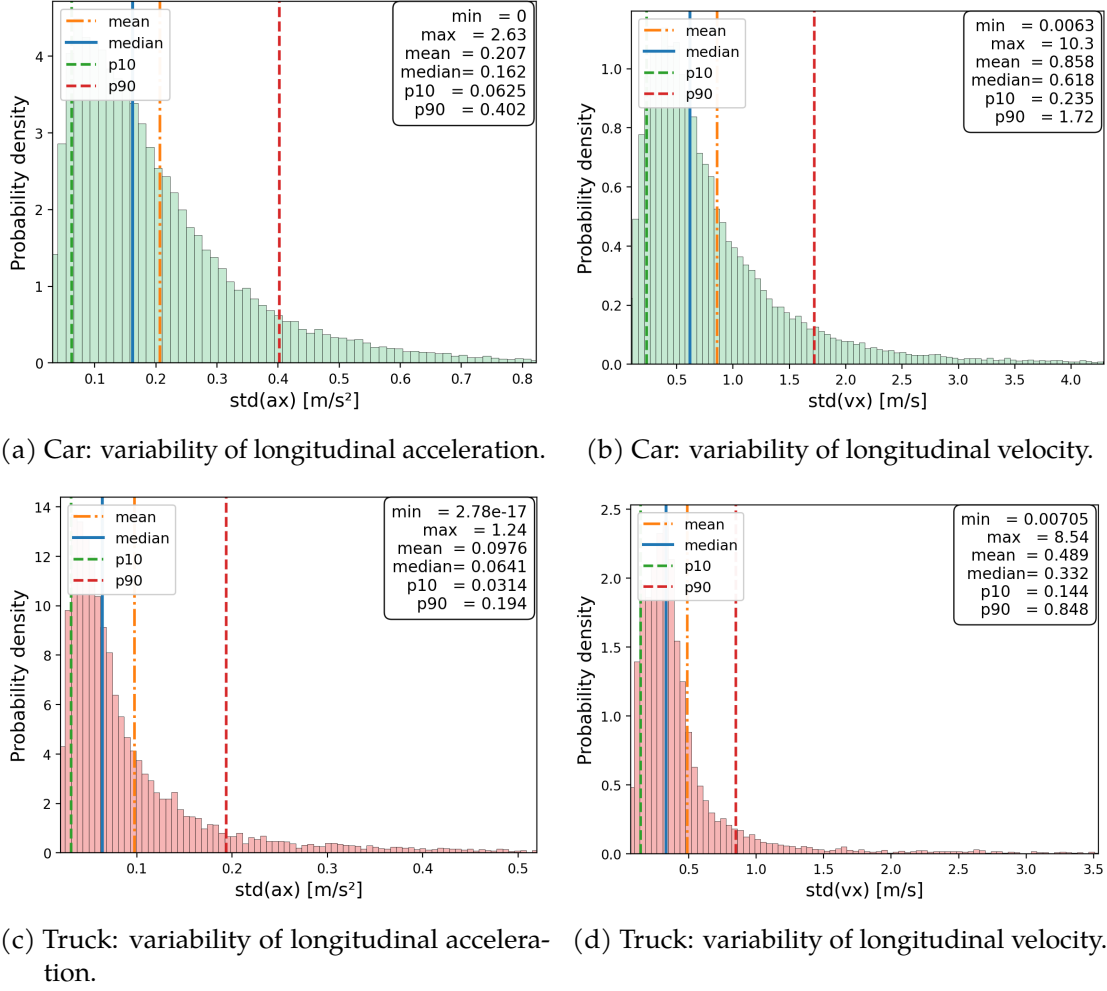
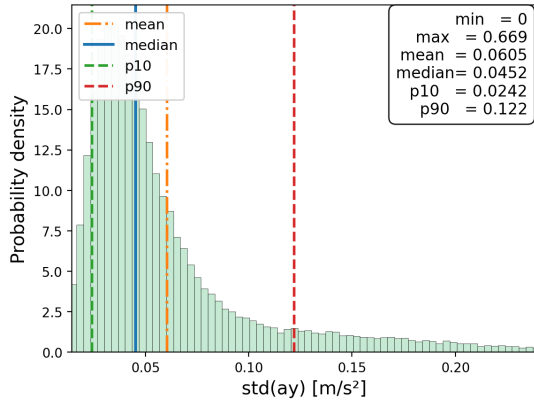
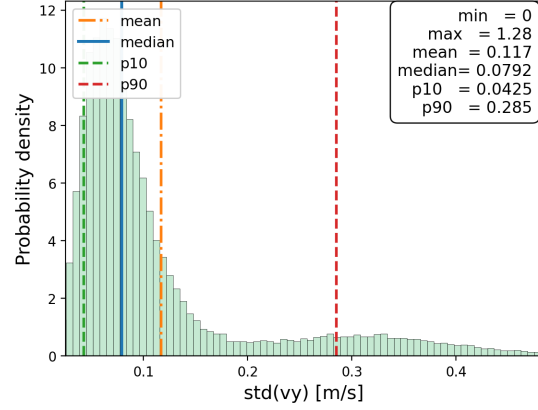


Figure B.2.: Class-wise trajectory-level variability distributions of the main kinematic variables.

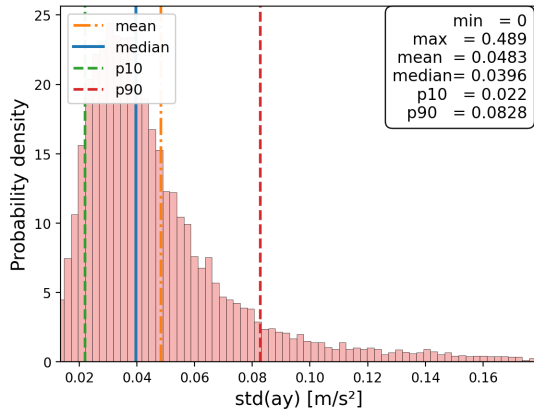
B. Class-Wise Distribution Plots



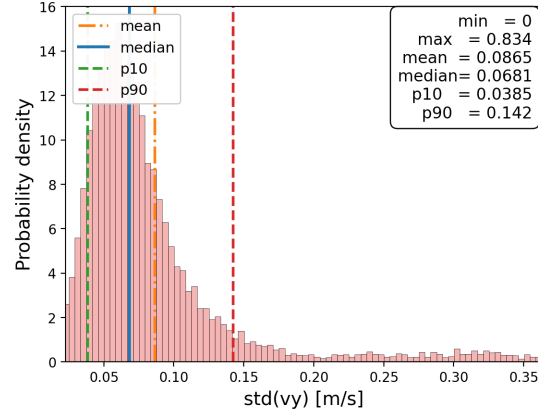
(e) Car: variability of lateral acceleration.



(f) Car: variability of lateral velocity.



(g) Truck: variability of lateral acceleration.



(h) Truck: variability of lateral velocity.

Figure B.2.: Class-wise trajectory-level variability distributions of the main kinematic variables (continued).

C. Complete Window-Level Observation Features

This appendix lists the complete set of window-level features used to construct the observation vector of the proposed model. The features are grouped according to their role in the representation.

Table C.1.: Window-level ego-motion features.

Window feature	Aggregation rule
<code>vx_last</code>	Most recent longitudinal velocity value in the window
<code>vx_slope</code>	Temporal slope of longitudinal velocity over the window
<code>ax_last</code>	Most recent longitudinal acceleration value in the window
<code>vy_last</code>	Most recent lateral velocity value in the window
<code>vy_slope</code>	Temporal slope of lateral velocity over the window
<code>ay_last</code>	Most recent lateral acceleration value in the window
<code>ax_neg_frac</code>	Fraction of frames in the window with negative longitudinal acceleration
<code>ax_pos_frac</code>	Fraction of frames in the window with positive longitudinal acceleration
<code>ax_zero_frac</code>	Fraction of frames in the window with zero longitudinal acceleration
<code>ay_neg_frac</code>	Fraction of frames in the window with negative lateral acceleration
<code>ay_pos_frac</code>	Fraction of frames in the window with positive lateral acceleration
<code>ay_zero_frac</code>	Fraction of frames in the window with zero lateral acceleration
<code>jerk_x_p95</code>	95th percentile of longitudinal jerk within the window
<code>jerk_y_p95</code>	95th percentile of lateral jerk within the window

Table C.2.: Window-level lane-change and lane-geometry features.

Window feature	Aggregation rule
<code>lc_left_present</code>	Binary indicator showing whether a left lane-change event occurs anywhere inside the window
<code>lc_right_present</code>	Binary indicator showing whether a right lane-change event occurs anywhere inside the window
<code>d_left_lane_last</code>	Most recent distance to the left lane boundary in the window
<code>d_left_lane_min</code>	Minimum distance to the left lane boundary within the window
<code>d_right_lane_last</code>	Most recent distance to the right lane boundary in the window
<code>d_right_lane_min</code>	Minimum distance to the right lane boundary within the window

C. Complete Window-Level Observation Features

Table C.3.: Window-level front-risk features.

Window feature	Aggregation rule
front_thw_last	Most recent valid THW value within the window
front_thw_vfrac	Fraction of frames in the window for which THW is valid
front_ttc_min	Minimum valid TTC value within the window
front_ttc_vfrac	Fraction of frames in the window for which TTC is valid

Table C.4.: Window-level features related to the front neighbor.

Window feature	Aggregation rule
front_dx_min	Minimum relative longitudinal distance to the front vehicle within the window
front_dvx_slope	Temporal slope of relative longitudinal velocity with respect to the front vehicle
front_exists_frac	Fraction of frames in the window in which a front vehicle is present

Table C.5.: Window-level features related to the side neighbors.

Window feature	Aggregation rule
left_side_dvx_last	Most recent relative longitudinal velocity with respect to the left-side neighbor
left_side_dvx_slope	Temporal slope of relative longitudinal velocity with respect to the left-side neighbor
left_side_dy_last	Most recent relative lateral position with respect to the left-side neighbor
left_side_exists_frac	Fraction of frames in the window in which a left-side neighbor is present
right_side_dvx_last	Most recent relative longitudinal velocity with respect to the right-side neighbor
right_side_dvx_slope	Temporal slope of relative longitudinal velocity with respect to the right-side neighbor
right_side_dy_last	Most recent relative lateral position with respect to the right-side neighbor
right_side_exists_frac	Fraction of frames in the window in which a right-side neighbor is present

C. Complete Window-Level Observation Features

Table C.6.: Window-level features related to the adjacent front neighbors.

Window feature	Aggregation rule
left_front_dx_last	Most recent relative longitudinal distance to the left-front neighbor
left_front_dvx_slope	Temporal slope of relative longitudinal velocity with respect to the left-front neighbor
left_front_dy_last	Most recent relative lateral position with respect to the left-front neighbor
left_front_exists_frac	Fraction of frames in the window in which a left-front neighbor is present
right_front_dx_last	Most recent relative longitudinal distance to the right-front neighbor
right_front_dvx_slope	Temporal slope of relative longitudinal velocity with respect to the right-front neighbor
right_front_dy_last	Most recent relative lateral position with respect to the right-front neighbor
right_front_exists_frac	Fraction of frames in the window in which a right-front neighbor is present

D. Emission Model Derivations

This appendix presents the theoretical derivations of the emission-model updates used in the proposed DBN. The derivation follows the standard EM formulation for latent-variable models [10], [72]. Exact implementation details and training-specific choices are described in the methodology chapter.

Let $O_{1:T} = (O_1, \dots, O_T)$ denote the observed sequence, and let $S_{1:T}$ and $A_{1:T}$ denote the corresponding latent style and action state sequences. The model parameters are collected in θ . Each observation at time step t is decomposed into a continuous part and a binary part:

$$O_t = (x_t, b_t) \quad (\text{D.1})$$

with

$$x_t \in \mathbb{R}^{D_c}, \quad (\text{D.2})$$

$$b_t \in \{0, 1\}^B \quad (\text{D.3})$$

In the E-step of the EM algorithm, the current parameter estimate θ^{old} is used to compute posterior probabilities over the latent variables [8], [10], [13].

The main posterior marginal used in the emission updates is:

$$\gamma_t(s, a) = p(S_t = s, A_t = a \mid O_{1:T}, \theta^{\text{old}}) \quad (\text{D.4})$$

This quantity is the posterior responsibility assigned to the latent pair (s, a) at time step t . For each fixed t , the values $\{\gamma_t(s, a)\}_{s,a}$ form a normalized joint posterior distribution:

$$\sum_{s,a} \gamma_t(s, a) = 1. \quad (\text{D.5})$$

The transition-update derivations also use the posterior transition marginals

$$\tilde{\xi}_t^S(s', s) = p(S_{t-1} = s', S_t = s \mid O_{1:T}, \theta^{\text{old}}), \quad t = 2, \dots, T, \quad (\text{D.6})$$

$$\tilde{\xi}_t^A(s, a', a) = p(S_t = s, A_{t-1} = a', A_t = a \mid O_{1:T}, \theta^{\text{old}}), \quad t = 2, \dots, T \quad (\text{D.7})$$

D.0.1. EM Objective

The goal of learning is to maximize the observed-data log-likelihood:

$$\log p(O_{1:T} \mid \theta) = \log \sum_{S_{1:T}, A_{1:T}} p(O_{1:T}, S_{1:T}, A_{1:T} \mid \theta) \quad (\text{D.8})$$

D. Emission Model Derivations

Direct maximization of (D.8) is difficult because the latent state trajectories are not observed and must be summed out [10], [72].

In the EM algorithm, this difficulty is addressed by introducing an auxiliary distribution $q(S_{1:T}, A_{1:T})$ over the latent trajectories, with:

$$q(S_{1:T}, A_{1:T}) \geq 0, \quad \sum_{S_{1:T}, A_{1:T}} q(S_{1:T}, A_{1:T}) = 1 \quad (\text{D.9})$$

Using this distribution, the log-likelihood can be rewritten as:

$$\log p(O_{1:T} | \theta) = \log \sum_{S_{1:T}, A_{1:T}} q(S_{1:T}, A_{1:T}) \frac{p(O_{1:T}, S_{1:T}, A_{1:T} | \theta)}{q(S_{1:T}, A_{1:T})} \quad (\text{D.10})$$

Jensen's inequality states that for a convex function f ,

$$f\left(\sum_i q_i x_i\right) \leq \sum_i q_i f(x_i), \quad q_i \geq 0, \quad \sum_i q_i = 1. \quad (\text{D.11})$$

Equivalently, if f is concave, the inequality direction is reversed. Since $\log(\cdot)$ is concave, applying Jensen's inequality to (D.10) gives:

$$\log p(O_{1:T} | \theta) \geq \sum_{S_{1:T}, A_{1:T}} q(S_{1:T}, A_{1:T}) \log \frac{p(O_{1:T}, S_{1:T}, A_{1:T} | \theta)}{q(S_{1:T}, A_{1:T})} \quad (\text{D.12})$$

Expanding the logarithm yields:

$$\begin{aligned} \log p(O_{1:T} | \theta) &\geq \sum_{S_{1:T}, A_{1:T}} q(S_{1:T}, A_{1:T}) \log p(O_{1:T}, S_{1:T}, A_{1:T} | \theta) \\ &\quad - \sum_{S_{1:T}, A_{1:T}} q(S_{1:T}, A_{1:T}) \log q(S_{1:T}, A_{1:T}) \end{aligned} \quad (\text{D.13})$$

The second term is the entropy of q . For fixed q , it does not depend on θ . Therefore, in the M-step it is sufficient to maximize only the first term [10], [72].

In EM, the auxiliary distribution is chosen as the posterior under the current parameter estimate:

$$q(S_{1:T}, A_{1:T}) = p(S_{1:T}, A_{1:T} | O_{1:T}, \theta^{\text{old}}) \quad (\text{D.14})$$

Substituting this choice into the lower bound gives the auxiliary function:

$$Q(\theta, \theta^{\text{old}}) = \mathbb{E}_{p(S_{1:T}, A_{1:T} | O_{1:T}, \theta^{\text{old}})} [\log p(O_{1:T}, S_{1:T}, A_{1:T} | \theta)] \quad (\text{D.15})$$

Thus, the E-step computes the posterior over the latent trajectories under θ^{old} , and the M-step updates θ by maximizing the expected complete-data log-likelihood $Q(\theta, \theta^{\text{old}})$ [8], [10], [13], [72].

D. Emission Model Derivations

Using the factorization of the proposed DBN, the complete-data distribution can be written as:

$$\begin{aligned}
 p(O_{1:T}, S_{1:T}, A_{1:T} \mid \theta) &= p(S_1 \mid \theta) p(A_1 \mid S_1; \theta) \prod_{t=2}^T p(S_t \mid S_{t-1}; \theta) \\
 &\quad \times \prod_{t=2}^T p(A_t \mid A_{t-1}, S_t; \theta) \prod_{t=1}^T p(O_t \mid S_t, A_t; \theta)
 \end{aligned} \tag{D.16}$$

Taking the logarithm converts the product into a sum:

$$\begin{aligned}
 \log p(O_{1:T}, S_{1:T}, A_{1:T} \mid \theta) &= \log p(S_1 \mid \theta) + \log p(A_1 \mid S_1; \theta) \\
 &\quad + \sum_{t=2}^T \log p(S_t \mid S_{t-1}; \theta) + \sum_{t=2}^T \log p(A_t \mid A_{t-1}, S_t; \theta) \\
 &\quad + \sum_{t=1}^T \log p(O_t \mid S_t, A_t; \theta)
 \end{aligned} \tag{D.17}$$

Substituting (D.17) into the auxiliary function (D.15) gives:

$$\begin{aligned}
 Q(\theta, \theta^{\text{old}}) &= \mathbb{E}_{p(S_{1:T}, A_{1:T} \mid O_{1:T}, \theta^{\text{old}})} [\log p(O_{1:T}, S_{1:T}, A_{1:T} \mid \theta)] \\
 &= \mathbb{E}_{p(S_{1:T}, A_{1:T} \mid O_{1:T}, \theta^{\text{old}})} \left[\log p(S_1 \mid \theta) + \log p(A_1 \mid S_1; \theta) \right. \\
 &\quad \left. + \sum_{t=2}^T \log p(S_t \mid S_{t-1}; \theta) + \sum_{t=2}^T \log p(A_t \mid A_{t-1}, S_t; \theta) \right. \\
 &\quad \left. + \sum_{t=1}^T \log p(O_t \mid S_t, A_t; \theta) \right]
 \end{aligned} \tag{D.18}$$

By linearity of expectation, this becomes

$$\begin{aligned}
 Q(\theta, \theta^{\text{old}}) &= \mathbb{E}[\log p(S_1 \mid \theta)] + \mathbb{E}[\log p(A_1 \mid S_1; \theta)] \\
 &\quad + \sum_{t=2}^T \mathbb{E}[\log p(S_t \mid S_{t-1}; \theta)] + \sum_{t=2}^T \mathbb{E}[\log p(A_t \mid A_{t-1}, S_t; \theta)] \\
 &\quad + \sum_{t=1}^T \mathbb{E}[\log p(O_t \mid S_t, A_t; \theta)],
 \end{aligned} \tag{D.19}$$

where all expectations are taken with respect to $p(S_{1:T}, A_{1:T} \mid O_{1:T}, \theta^{\text{old}})$ [10], [72].

These expectations can be written in terms of the posterior marginals from the E-step. For the initial style term, define:

$$\gamma_1^S(s) = p(S_1 = s \mid O_{1:T}, \theta^{\text{old}}) = \sum_a \gamma_1(s, a) \tag{D.20}$$

Using (D.4), (D.6), and (D.7), the auxiliary function becomes:

$$\begin{aligned}
Q(\theta, \theta^{\text{old}}) = & \sum_s \gamma_1^S(s) \log p(S_1 = s \mid \theta) + \sum_{s,a} \gamma_1(s,a) \log p(A_1 = a \mid S_1 = s; \theta) \\
& + \sum_{t=2}^T \sum_{s',s} \xi_t^S(s',s) \log p(S_t = s \mid S_{t-1} = s'; \theta) \\
& + \sum_{t=2}^T \sum_{s,a',a} \xi_t^A(s,a',a) \log p(A_t = a \mid A_{t-1} = a', S_t = s; \theta) \\
& + \sum_{t=1}^T \sum_{s,a} \gamma_t(s,a) \log p(O_t \mid S_t = s, A_t = a; \theta)
\end{aligned} \tag{D.21}$$

This is the expected complete-data log-likelihood written explicitly in terms of the posterior quantities computed in the E-step [8], [13], [72].

For the emission-parameter derivations in this appendix, only the last term in (D.21) is required. Therefore, the emission contribution is:

$$Q_{\text{emit}}(\theta, \theta^{\text{old}}) = \sum_{t=1}^T \sum_{s,a} \gamma_t(s,a) \log p(O_t \mid S_t = s, A_t = a; \theta) \tag{D.22}$$

The following sections specialize (D.22) to the hierarchical and PoE emission formulations considered in this work.

D.1. Hierarchical Emission Model

For the hierarchical emission formulation introduced in Section 4.4, the emission model is parameterized directly for each joint latent-state pair (s, a) . The corresponding M-step therefore follows the standard EM pattern for latent-variable models, where each parameter block is updated using posterior weights [72], [73].

Using the decomposition $O_t = (x_t, b_t)$, the emission factorizes as:

$$p(O_t \mid s, a; \theta_{s,a}) = p(x_t \mid s, a; \theta_{s,a}^{(x)}) p(b_t \mid s, a; \theta_{s,a}^{(b)}), \tag{D.23}$$

with

$$\theta_{s,a}^{(x)} = (\mu_{s,a}, \Sigma_{s,a}), \quad \theta_{s,a}^{(b)} = \{p_{s,a,j}\}_{j=1}^B \tag{D.24}$$

For the continuous features, a Gaussian distribution with diagonal covariance is used:

$$p(x_t \mid s, a) = \mathcal{N}(x_t \mid \mu_{s,a}, \Sigma_{s,a}), \quad \Sigma_{s,a} = \text{diag}(\sigma_{s,a,1}^2, \dots, \sigma_{s,a,D_c}^2) \tag{D.25}$$

For the binary features, a factorized Bernoulli distribution is used:

$$p(b_t \mid s, a) = \prod_{j=1}^B (p_{s,a,j})^{b_t^{(j)}} (1 - p_{s,a,j})^{1-b_t^{(j)}} \tag{D.26}$$

D. Emission Model Derivations

Hence, for a fixed joint state pair (s, a) , the log-emission probability becomes:

$$\begin{aligned} \log p(O_t | s, a; \theta_{s,a}) &= \log p(x_t | s, a) + \log p(b_t | s, a) \\ &= \log \mathcal{N}(x_t | \mu_{s,a}, \Sigma_{s,a}) + \sum_{j=1}^B \left[b_t^{(j)} \log p_{s,a,j} + (1 - b_t^{(j)}) \log(1 - p_{s,a,j}) \right] \end{aligned} \quad (\text{D.27})$$

Substituting (D.27) into the emission contribution of the auxiliary function, defined in (D.22), gives:

$$\begin{aligned} Q_{\text{emit}}(\theta, \theta^{\text{old}}) &= \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \log \mathcal{N}(x_t | \mu_{s,a}, \Sigma_{s,a}) \\ &\quad + \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \sum_{j=1}^B \left[b_t^{(j)} \log p_{s,a,j} + (1 - b_t^{(j)}) \log(1 - p_{s,a,j}) \right] \end{aligned} \quad (\text{D.28})$$

For convenience, define the effective posterior count of the joint state pair (s, a) as

$$N_{s,a} = \sum_{t=1}^T \gamma_t(s, a). \quad (\text{D.29})$$

D.1.1. Parameter Updates

The emission parameters are obtained by maximizing (D.28) with respect to the parameters of each joint state pair (s, a) . Because the parameters for one joint state do not appear in the emission terms of any other joint state, the optimization separates across (s, a) and yields closed-form updates [72], [73].

Gaussian mean. For fixed covariance, the part of the objective that depends on $\mu_{s,a}$ is:

$$-\frac{1}{2} \sum_{t=1}^T \gamma_t(s, a) (x_t - \mu_{s,a})^\top \Sigma_{s,a}^{-1} (x_t - \mu_{s,a}) \quad (\text{D.30})$$

Differentiating with respect to $\mu_{s,a}$ and setting the derivative to zero yields:

$$\mu_{s,a} = \frac{\sum_{t=1}^T \gamma_t(s, a) x_t}{\sum_{t=1}^T \gamma_t(s, a)} = \frac{1}{N_{s,a}} \sum_{t=1}^T \gamma_t(s, a) x_t \quad (\text{D.31})$$

Gaussian covariance. Assuming a diagonal covariance matrix, each variance parameter can be optimized independently. For dimension d , the relevant term is

$$-\frac{1}{2} \sum_{t=1}^T \gamma_t(s, a) \left[\log \sigma_{s,a,d}^2 + \frac{(x_t^{(d)} - \mu_{s,a}^{(d)})^2}{\sigma_{s,a,d}^2} \right]. \quad (\text{D.32})$$

D. Emission Model Derivations

Differentiating with respect to $\sigma_{s,a,d}^2$ and setting the result to zero gives:

$$\sigma_{s,a,d}^2 = \frac{1}{N_{s,a}} \sum_{t=1}^T \gamma_t(s,a) (x_t^{(d)} - \mu_{s,a}^{(d)})^2 \quad (\text{D.33})$$

Hence,

$$\Sigma_{s,a} = \text{diag}(\sigma_{s,a,1}^2, \dots, \sigma_{s,a,D_c}^2). \quad (\text{D.34})$$

Bernoulli parameters. For each binary feature j , the Bernoulli part of the objective is

$$\sum_{t=1}^T \gamma_t(s,a) \left[b_t^{(j)} \log p_{s,a,j} + (1 - b_t^{(j)}) \log(1 - p_{s,a,j}) \right]. \quad (\text{D.35})$$

Differentiating with respect to $p_{s,a,j}$ and setting this derivative to zero gives:

$$p_{s,a,j} = \frac{\sum_{t=1}^T \gamma_t(s,a) b_t^{(j)}}{\sum_{t=1}^T \gamma_t(s,a)} = \frac{1}{N_{s,a}} \sum_{t=1}^T \gamma_t(s,a) b_t^{(j)} \quad (\text{D.36})$$

Thus, the hierarchical emission model admits closed-form M-step updates, where each parameter is estimated as a posterior-weighted maximum-likelihood quantity for the corresponding joint latent-state pair [72], [73].

D.2. Product-of-Experts Emission Model

For the PoE emission formulation introduced in Section 4.4, the emission distribution for the joint latent-state pair (s,a) is constructed by multiplying a style-dependent expert and an action-dependent expert, followed by normalization:

$$p(O_t | S_t = s, A_t = a) = \frac{p_s(O_t | s) p_a(O_t | a)}{Z_{s,a}} \quad (\text{D.37})$$

This is the standard product-of-experts construction, in which several simpler expert models are combined multiplicatively and then renormalized to obtain a valid probability distribution [74].

Here, $Z_{s,a}$ is the normalization constant that ensures that (D.37) defines a valid probability distribution. Using the continuous-binary decomposition of the observation, it can be written more explicitly as

$$Z_{s,a} = \sum_{b \in \{0,1\}^B} \int p_s(x, b | s) p_a(x, b | a) dx. \quad (\text{D.38})$$

Using the decomposition $O_t = (x_t, b_t)$, both experts factorize into continuous and binary parts:

$$p_s(O_t | s) = p_s(x_t | s) p_s(b_t | s), \quad (\text{D.39})$$

$$p_a(O_t | a) = p_a(x_t | a) p_a(b_t | a). \quad (\text{D.40})$$

D. Emission Model Derivations

The corresponding parameter sets are written as

$$\theta_s = (\mu_s, \Sigma_s, \{p_j^s\}_{j=1}^B), \quad \theta_a = (\mu_a, \Sigma_a, \{p_j^a\}_{j=1}^B). \quad (\text{D.41})$$

For the continuous features, Gaussian experts with diagonal covariance are used:

$$p_s(x_t | s) = \mathcal{N}(x_t | \mu_s, \Sigma_s), \quad \Sigma_s = \text{diag}(\sigma_{s,1}^2, \dots, \sigma_{s,D_c}^2), \quad (\text{D.42})$$

$$p_a(x_t | a) = \mathcal{N}(x_t | \mu_a, \Sigma_a), \quad \Sigma_a = \text{diag}(\sigma_{a,1}^2, \dots, \sigma_{a,D_c}^2). \quad (\text{D.43})$$

For the binary features, factorized Bernoulli experts are used:

$$p_s(b_t | s) = \prod_{j=1}^B (p_j^s)^{b_t^{(j)}} (1 - p_j^s)^{1-b_t^{(j)}}, \quad (\text{D.44})$$

$$p_a(b_t | a) = \prod_{j=1}^B (p_j^a)^{b_t^{(j)}} (1 - p_j^a)^{1-b_t^{(j)}}. \quad (\text{D.45})$$

Because the experts factorize into continuous and binary parts, the normalization constant also factorizes:

$$\begin{aligned} Z_{s,a} &= \sum_{b \in \{0,1\}^B} \int p_s(x, b | s) p_a(x, b | a) dx \\ &= \left(\int p_s(x | s) p_a(x | a) dx \right) \left(\sum_{b \in \{0,1\}^B} p_s(b | s) p_a(b | a) \right) \\ &= Z_{s,a}^{(x)} Z_{s,a}^{(b)}. \end{aligned} \quad (\text{D.46})$$

The continuous normalization term is therefore

$$Z_{s,a}^{(x)} = \int \mathcal{N}(x | \mu_s, \Sigma_s) \mathcal{N}(x | \mu_a, \Sigma_a) dx. \quad (\text{D.47})$$

Using the standard Gaussian product identity, this becomes

$$Z_{s,a}^{(x)} = \mathcal{N}(\mu_s | \mu_a, \Sigma_s + \Sigma_a). \quad (\text{D.48})$$

Hence,

$$\begin{aligned} \log Z_{s,a}^{(x)} &= -\frac{D_c}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma_s + \Sigma_a| \\ &\quad - \frac{1}{2} (\mu_s - \mu_a)^\top (\Sigma_s + \Sigma_a)^{-1} (\mu_s - \mu_a). \end{aligned} \quad (\text{D.49})$$

For the binary part,

$$Z_{s,a}^{(b)} = \sum_{b \in \{0,1\}^B} \prod_{j=1}^B (p_j^s)^{b^{(j)}} (1 - p_j^s)^{1-b^{(j)}} (p_j^a)^{b^{(j)}} (1 - p_j^a)^{1-b^{(j)}}. \quad (\text{D.50})$$

D. Emission Model Derivations

Since the Bernoulli experts factorize across dimensions, the sum also factorizes:

$$Z_{s,a}^{(b)} = \prod_{j=1}^B \left[p_j^s p_j^a + (1 - p_j^s)(1 - p_j^a) \right]. \quad (\text{D.51})$$

Taking the logarithm yields

$$\log Z_{s,a}^{(b)} = \sum_{j=1}^B \log \left[p_j^s p_j^a + (1 - p_j^s)(1 - p_j^a) \right]. \quad (\text{D.52})$$

Combining the two experts and the normalization term, the log-emission probability becomes

$$\begin{aligned} \log p(O_t | s, a) &= \log p_s(x_t | s) + \log p_a(x_t | a) + \log p_s(b_t | s) + \log p_a(b_t | a) \\ &\quad - \log Z_{s,a}^{(x)} - \log Z_{s,a}^{(b)}. \end{aligned} \quad (\text{D.53})$$

Substituting (D.53) into the emission contribution of the auxiliary function, defined in (D.22), gives

$$\begin{aligned} Q_{\text{emit}}(\theta, \theta^{\text{old}}) &= \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \log \mathcal{N}(x_t | \mu_s, \Sigma_s) \\ &\quad + \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \log \mathcal{N}(x_t | \mu_a, \Sigma_a) \\ &\quad + \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \sum_{j=1}^B \left[b_t^{(j)} \log p_j^s + (1 - b_t^{(j)}) \log(1 - p_j^s) \right] \\ &\quad + \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \sum_{j=1}^B \left[b_t^{(j)} \log p_j^a + (1 - b_t^{(j)}) \log(1 - p_j^a) \right] \\ &\quad - \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \log Z_{s,a}^{(x)} - \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \log Z_{s,a}^{(b)}. \end{aligned} \quad (\text{D.54})$$

D.2.1. Implication for the M-Step

In contrast to the hierarchical emission model, the PoE objective does not separate into independent parameter blocks for each joint state pair. The reason is that the normalization terms $\log Z_{s,a}^{(x)}$ and $\log Z_{s,a}^{(b)}$ depend on the same expert parameters that also appear in the data-fit terms. As a result, the style and action experts are coupled through the partition function [74].

More specifically, any update of a style-expert parameter affects not only the style likelihood term, but also all normalization terms involving that style state across the action experts. The same holds symmetrically for the action-expert parameters. Therefore,

the emission M-step no longer reduces to a set of independent closed-form weighted maximum-likelihood updates.

At the theoretical level, this means that the PoE emission parameters are optimized by increasing

$$Q_{\text{emit}}(\theta, \theta^{\text{old}}) \quad (\text{D.55})$$

with respect to the expert parameters using a generalized EM step, rather than by solving closed-form maximization equations for each parameter separately [72], [75].

D.2.2. Gradient Form

Let ϑ_s denote a parameter of the style expert and ϑ_a a parameter of the action expert. Differentiating (D.54) gives the generic gradient structure

$$\frac{\partial Q_{\text{emit}}}{\partial \vartheta_s} = \sum_{t=1}^T \sum_a \gamma_t(s, a) \left[\frac{\partial}{\partial \vartheta_s} \log p_s(O_t | s) - \frac{\partial}{\partial \vartheta_s} \log Z_{s,a} \right], \quad (\text{D.56})$$

and similarly

$$\frac{\partial Q_{\text{emit}}}{\partial \vartheta_a} = \sum_{t=1}^T \sum_s \gamma_t(s, a) \left[\frac{\partial}{\partial \vartheta_a} \log p_a(O_t | a) - \frac{\partial}{\partial \vartheta_a} \log Z_{s,a} \right]. \quad (\text{D.57})$$

These expressions show that each gradient contains two competing contributions: a data-fit term from the expert likelihood and a normalization term arising from the partition function. The second term is specific to the PoE formulation and is the reason why the emission M-step does not admit the same closed-form updates as in the hierarchical case [74].

Declaration of Compliance

I hereby declare to have written this work independently and to have respected in its preparation the relevant provisions, in particular those corresponding to the copyright protection of external materials. Whenever external materials (such as images, drawings, text passages) are used in this work, I declare that these materials are referenced accordingly (e. g. quote, source) and, whenever necessary, consent from the author to use such materials in my work has been obtained.

Signature:

Stuttgart, on the 15 April 2026